IST3134 BIG DATA ANALYTICS IN THE CLOUD

GROUP ASSIGNMENT

TOPIC			
Cloud-Based Big Data Analysis of NYC TLC Taxi Trip Record Data			
NO.	**MEMBER**	**STUDENT ID**	**CONTRIBUTION**
1	Yeoh Jie Long	22071112	Spark SQL/Py Spark/AWS
2	Ooi Jin Yon	22039069	Python pandas

Table of Contents

1.0 Introduction 4
1.1 Background 4
1.2 Problem Statement 4
1.3 Research Questions 5
1.4 Objectives 5
1.5 Scope and Exclusions 5
2.0 Dataset Introduction 6
2.1 Dataset Description 6
2.2 Dataset Characteristics 6
2.3 Key Variables 9
2.4 Raw Data Quality Concerns and Limitations 10
3.0 Methodology 11
3.1 Overall Research Design 11
3.2 Platform Architecture 11
3.3 Data Acquisition and Storage Method 12
3.4 Data Profiling and Preprocessing Method 13
3.4.1 Raw Data Profiling 13
3.4.2 Schema Standardisation 13
3.4.3 Structural Cleaning 13
3.4.4 Analysis-Specific Validity Flags 14
3.4.5 Feature Engineering 14
3.4.6 Taxi Zone Enrichment 14
3.4.7 Processed Data Storage 15
3.4.8 Centralised Dataset Consistency 15
3.5 Spark SQL and PySpark Analytical Method 15
3.6 pandas Analytical Method 16
3.7 Analytical Plan 17
3.8 Validation and Benchmark Design 17
3.9 Visualisation Method 18
4.0 Implementation 18
4.1 GitHub Repository and Collaboration 18
4.2 Amazon S3 Organisation 19
4.3 AWS Cluster Configuration and EC2 Hadoop/YARN Implementation 21
4.4 PySpark Preprocessing and Centralised Dataset Implementation 23
4.5 Spark SQL Analysis Implementation 24

4.6 Spark Performance and Physical Plan Implementation 25
4.6.1 Matched Spark-pandas Scalability Benchmark 27
4.7 pandas Preprocessing and Analysis Implementation 29
4.8 Visualisation Implementation 30
4.9 Testing and Reproducibility 31
5.0 Result and Analysis 32
5.1 Dataset Volume and Raw Quality 32
5.2 Preprocessing Outcomes 33
5.3 Hourly Demand Patterns 34
5.4 Pickup Zones and Boroughs 34
5.5 Fare Patterns by Distance Band 36
5.6 Passenger-Count Patterns 39
5.7 Weekday versus Weekend 41
5.8 2024 versus 2025 Comparison 42
5.9 Integrated Findings 44
6.0 Comparison 45
6.1 Correctness Comparison 45
6.2 Runtime and Scalability 46
6.3 Memory and Resource Feasibility 47
6.4 Development and Operational Complexity 49
6.5 Suitable Use Cases 50
7.0 Discussion 51
7.1 Interpretation of Findings 51
7.2 Advantages of Big Data Analytics in the Cloud 51
7.3 Limitations and Threats to Validity 52
7.4 Recommendations and Future Improvements 52
8.0 Conclusion 53
8.1 Achievement of Objectives 53
8.2 Final Conclusion 53
9.0 Individual Reflection 54
9.1 Yeoh Jie Long 54
9.2 Ooi Jin Yon 56
10.0 Reference 57
11.0 Appendix 57

1.0 Introduction

1.1 Background

The rapid growth of digital technologies has resulted in the generation of massive volumes of data from various industries, which include transportation, healthcare, finance and social media. The current traditional data processing tools always have issue to analyse large-scale datasets because there are limitations in storage capacity, memory and computational performance. Thus, big data analytics has become more important as it help to get the meaningful insight from complex datasets. Moreover, cloud computing enhances big data analytics by providing scalable computing resources that help organisations to process large datasets efficiently without buying expensive hardware infrastructure.

Apache Spark is one of the famous big data frameworks. It is an open-source distributed computing platform designed for high-speed data processing. It is a general-purpose cluster computing framework that supports scalable data analytics through in-memory processing and provides integrated libraries for structured data processing, machine learning, graph analysis and stream processing (Salloum et al., 2016). Unlike traditional tools, it distributes computations across multiple computing nodes. This made it become efficient in process large datasets with better scalability and performance. In addition, Spark SQL improves the capabilities by allowing the structured and semi-structured data with SQL-like syntax. These made Apache Spark a leading platform for cloud-based big data analysis in academy and industry (Salloum et al., 2016)

1.2 Problem Statement

The transportation industry had collects large volumes of operational data everyday from various type of transport services, which include taxi. The New York City Taxi and Limousine Commission (NYC TLC) had published millions of taxi trip records every month. From the dataset, it provides detailed information like pickup times, drop-off times, locations, trip distances, passenger counts, fare amounts and payment details. By analysing these data, it will provide some valuable insights into taxi demand, travel behaviour, fare patterns and trends that can be used.

However, processing multiple months of NYC TLC taxi trip records using traditional single-machine tools can be computationally expensive and memory-intensive. As the dataset continue grows, the processing time will increase significantly. This will make it difficult to perform efficient large-scale analysis. Therefore, a distributed computing is required to process and analyse the data more efficiently in a cloud environment.

This project will use Apache Spark SQL to analyse multiple months of NYC Yellow Taxi Trip records to identify taxi demand patterns, peak travel periods, fare trends and travel

behaviour. The performance of Apache Spark SQL is also compared with a traditional implementation to demonstrate the advantages of Big Data Analytics based on scalability, processing efficiency, and suitability for analysing large datasets in a cloud computing environment.

1.3 Research Questions

1. How large is the selected dataset, and what data-quality issues are present?
2. Which pickup hours, days and months record the highest normalised Yellow Taxi demand?
3. Which pickup zones and boroughs generate the highest number of trips?
4. How do fares, total amounts, durations and fare per mile vary across trip-distance bands?
5. What passenger-count patterns are present, and how are they associated with fare and distance?
6. How do weekday and weekend demand and trip characteristics differ?
7. What descriptive differences can be observed between matching periods in 2024 and 2025, including the CBD congestion-fee field?
8. How do Spark SQL/PySpark and pandas compare in correctness, runtime growth, scalability and suitability?

1.4 Objectives

The objectives of this project are:

1. Build a reproducible cloud-based pipeline that processes the complete selected dataset using Spark on AWS Academy cloud infrastructure with Amazon S3, Hadoop/HDFS and YARN.
2. Profile, validate and clean the source records using documented analysis-specific rules.
3. Convert numeric location identifiers into meaningful pickup-zone and borough names using the official lookup table.
4. Analyse temporal demand, location concentration, distance–fare relationships, passenger patterns, weekday/weekend differences and 2024–2025 differences.
5. Compare Spark and pandas using January 2024 as the common correctness-validation dataset and progressively larger 1, 3, 6, 12 and 24-month workloads for scalability.
6. Publish source code, execution instructions, small outputs and evidence through GitHub without exposing credentials or large raw files.

1.5 Scope and Exclusions

The project includes 24 monthly Yellow Taxi Parquet files from January 2024 to December 2025 and one Taxi Zone Lookup CSV. The main platform is AWS Academy Learner Lab, which consists of Amazon S3 and a self-managed EC2 Hadoop/HDFS/YARN cluster with

PySpark and Spark SQL. The comparison platform will be local Python with pandas, PyArrow and Matplotlib. Green Taxi, FHV, High Volume FHV, streaming, machine learning, route prediction and cross-cloud comparisons are not included.

2.0 Dataset Introduction

2.1 Dataset Description

The dataset we used is the NYC Taxi Trip Record Data that published by the New York City Taxi and Limousine Commission (NYC TLC). The dataset contains detailed records of taxi trips in New York City, including pickup and drop-off timestamps, trip distance, passenger count, fare amount, payment type and pickup and drop-off location identifiers.

NYC TLC releases data monthly in Parquet format. Parquet is a new columnar storage format that provides better storage efficiency and query performance for analytical workloads (Apache Software Foundation, 2024). Parquet stores data by column versus the row-based approach of traditional formats, enabling analytical engines like Spark SQL to only read the columns a query needs. This can alleviate unnecessary disk I/O, and facilitate efficient analytical processing (Armbrust et al., 2015; Zeng et al., 2023). The analysis for this project is limited to Yellow Taxi Trip Records from 2024 to 2025, which allows for a descriptive comparison of taxi demand and fare patterns between these two years of the study.

2.2 Dataset Characteristics

Using spark profiling, it was determined that the selected workload is exactly 24 Yellow Taxi Parquet files, between January 2024 and December 2025. The files contained 89,892,322 raw trip records in total: 41,169,720 records in 2024 and 48,722,602 records in 2025. Monthly raw volume ranged from 2,964,624 records in January 2024 to 4,591,845 records in May 2025. This scale had to go through repeated schema inspection, transformation, filtering, joining and aggregating many millions of rows, thus fulfilling the requirement of the assignment to use a large enough scale.

Two stable schema variants were found. There are 19 columns in all 2024 monthly files and 20 columns in all 2025 monthly files, due to the schema change of the cbd_congestion_fee field added in 2025. The Taxi Zone Lookup had 265 rows and 265 different LocationID values, and there were no repeated or null LocationID values. This clean lookup made it suitable for deterministic pickup-zone enrichment.

Measured characteristic	Spark profiling result
Selected period	January 2024 - December 2025
Monthly Parquet files	24
Total raw rows	89,892,322
2024 raw rows	41,169,720
2025 raw rows	48,722,602
2024 schema	19 columns
2025 schema	20 columns
Additional 2025 field	cbd_congestion_fee
Taxi Zone Lookup	265 rows; 265 distinct IDs; 0 duplicates; 0 null IDs

Table 2.2.1 Spark Dataset Profiling Summary

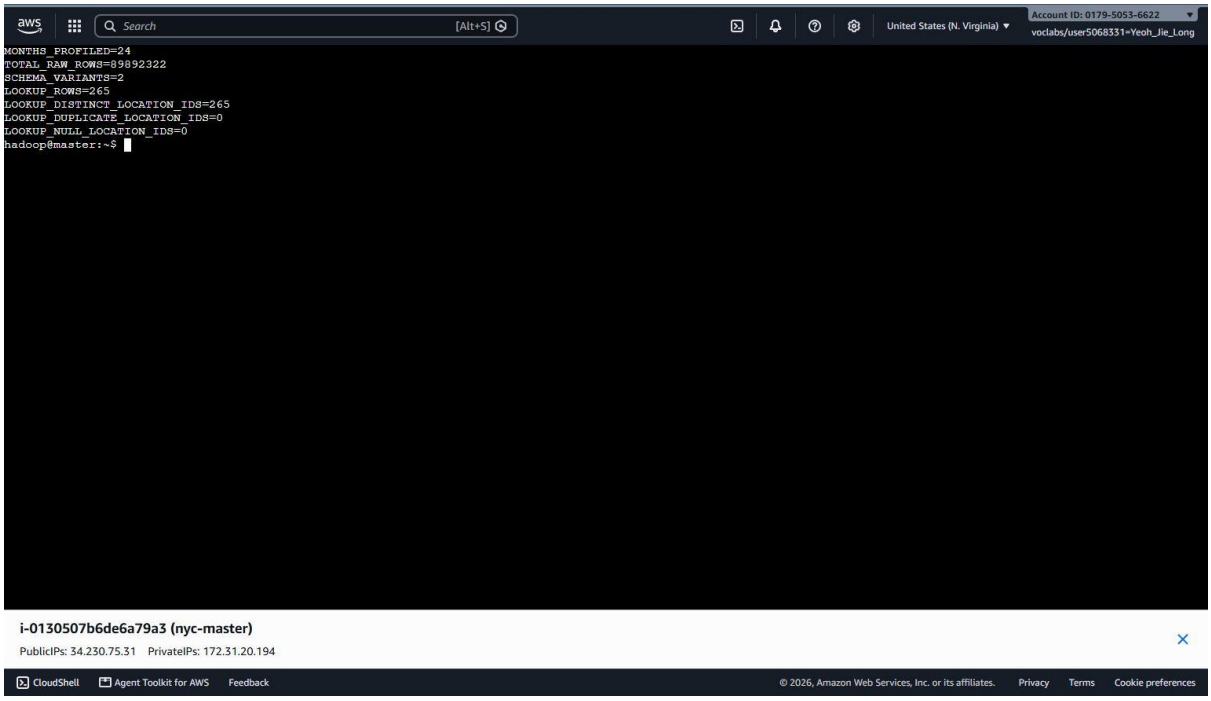


Figure 2.2.1 Spark raw-data profiling summary for the 24-month dataset.

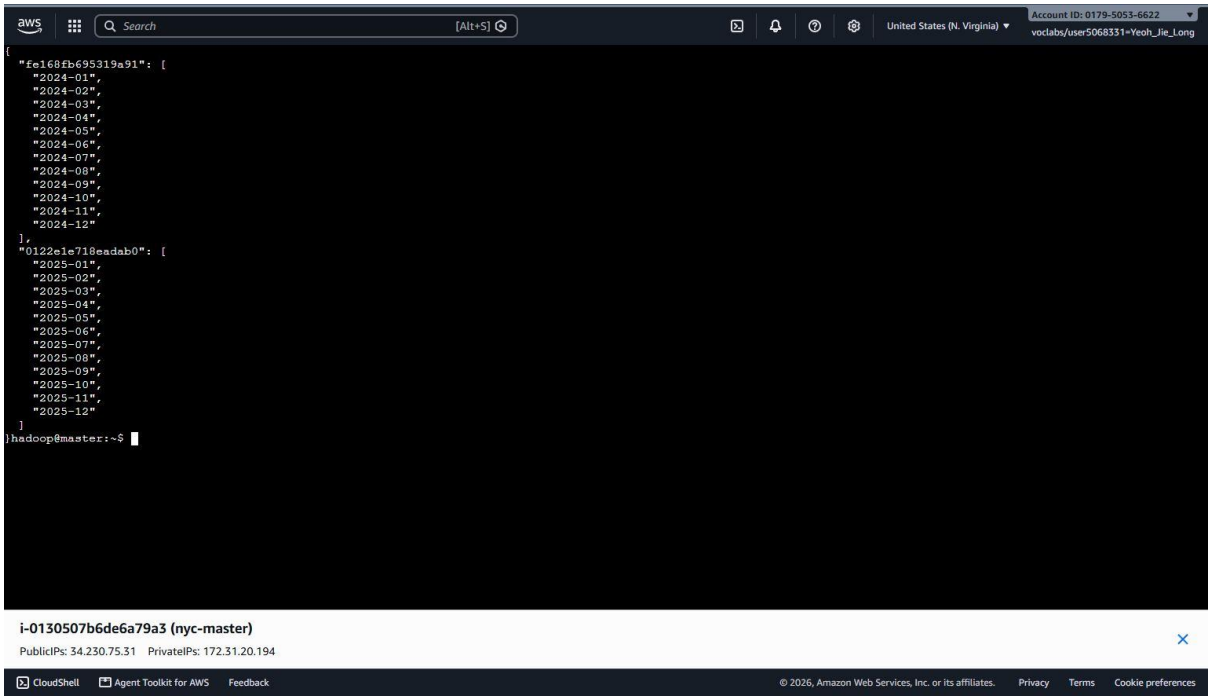


Figure 2.2.2 Two raw schema variants identified across 2024 and 2025

Characteristic	Description
Dataset Name	NYC TLC Taxi Trip Record Data
Data Provider	New York City Taxi and Limousine Commission
Dataset Type	Public transportation data
Dataset Link	https://www.nyc.gov/site/tlc/about/tlc-trip-record-data.page
File Format	Parquet
Taxi Type	Yellow Taxi
Time Period	2024 and 2025
Data Size	89,892,322 raw trip records across 24 monthly Parquet files
Processing Platform	AWS Academy: S3 + EC2 Hadoop/HDFS/YARN + PySpark/Spark SQL
Reference Data	Taxi Zone Lookup CSV (265 rows; 265 distinct LocationID values)

Table 2.2.2 Dataset Characteristics

2.3 Key Variables

The source variables were chosen based on the research questions for demand, location, distance, fare, passenger behaviour and comparison for 2024-2025. In addition to the variables in Table 2.3.1, the Spark pipeline standardised fee fields and created derived variables needed for consistent grouping and validation. Importantly, the 2024 cbd_congestion_fee was not set to zero, but rather to a typed null, to avoid misrepresenting historical unavailability as zero congestion fee.

Additional / derived variable	Purpose
tip_amount	Tip recorded in the TLC trip record; used only with appropriate interpretation.
payment_type	Payment-method code used for payment-type summaries.
Airport_fee / airport_fee	Case-normalised optional airport fee field.
cbd_congestion_fee	2025 CBD congestion-fee field; typed null when unavailable in 2024.
pickup_year / pickup_month	Derived partition and temporal grouping fields.
pickup_date / pickup_hour / weekend	Derived demand-analysis fields.
trip_duration_min	Drop-off minus pickup time in minutes.
valid_distance / valid_fare	Analysis-specific flags; non-positive values are retained in the base dataset.
distance_band	0-<2, 2-<5, 5-<10, 10-<20, 20+, or Invalid/Non-positive.
passenger_group	Missing, 0, 1-6 or 7+.
pickup_Borough / pickup_Zone / pickup_service_zone	Pickup attributes from the Taxi Zone Lookup left join.

Table 2.3.1 Additional and derived variables used in the analysis

Variable	Description
tpep_pickup_datetime	Pickup date and time
tpep_dropoff_datetime	Drop-off date and time
PULocationID	Pickup location ID
DOLocationID	Dropoff location ID
passenger_count	Number of passengers

Variable	Description
trip_distance	Distance travelled (in miles)
fare_amount	Taxi fare
total_amount	Total trip payment amount
congestion_surcharge	Congestion pricing surcharge

Table 2.3.2 Selected source variables

2.4 Raw Data Quality Concerns and Limitations

The NYC Yellow Taxi dataset contains several data-quality issues that are typical of operational data, including missing passenger counts, zero or negative trip distances, zero or negative fares, invalid timestamps, and pickup dates outside the expected source month. Potential unmatched location identifiers were also checked during enrichment; however, the final pickup-location join produced zero unmatched rows.

A limitation of the dataset is that tip amounts are only reliably recorded for credit card transactions because the cash tips are not captured by the TLC system. Furthermore, the data does not contain a unique identifier that is always reliable, making duplicate detection challenging. Finally, the dataset is observational in nature. As a result, the analyses reveal patterns and associations, but do not prove causal relationships.

The raw Spark profile also showed clear timestamp anomalies. A few monthly files included historical dates like 2002 or 2007, 2008 or 2009, and some data spanned beyond the intended source month. For instance, in June 2024 raw file, pickup timestamps were included for up to June 2026, and December 2025 drop-off timestamps were included for up to January 2026. These observations suggested a need for an explicit source-month check and a maximum trip-duration rule instead of making the assumption that all timestamps in a monthly file were from that month.

Quality diagnostics were determined prior to structural filtering and were structured intentionally to be non-exhaustive; they could be applied to one trip more than once on different quality issues. Across all 24 files, 559,814 records had drop-off times that were not after pickup, 40,094 had durations above 240 minutes, 634 had pickup timestamps outside the source month, 2,179,263 had null/non-positive distance, 3,618,518 had null/non-positive fare, and 15,703,126 had missing passenger counts. The pipeline thus did not delete any anomalous records from a single universal set of data but instead separated out the structural validity from the analysis-specific fare, distance, and passenger validity.

3.0 Methodology

3.1 Overall Research Design

The current study uses a quantitative descriptive and comparative research design for the analysis of NYC Yellow Taxi Trip Records of January 2024 through December 2025. The descriptive element looks at demand, pickup sites, distance and fare patterns, passenger-count patterns, and weekday/weekend travel behaviour. The comparative is comparing between similar periods of 2024 and 2025, and comparing Apache Spark vs pandas with the same analytical definitions. The AWS Academy EC2 Hadoop/HDFS/YARN cluster consists of two nodes, with Spark being used as the primary full-data processing platform, and pandas being used locally as an independent comparison implementation.

The 2024-versus-2025 comparison is descriptive and observational. The differences in trip volume, fare-related measures or the 2025 CBD congestion-fee field are therefore reported as associations in the observed data, and are not interpreted as evidence that the congestion pricing caused the differences.

January 2024 is used as the common validation period for correctness validation to allow Spark and pandas to be validated against the same source month (january 2024) and structural rules. The scalability benchmark then moves on to increasingly larger common periods of 1, 3, 6, 12 and 24 months. The overall analytical areas include dataset quality, temporal demand, pickup zones, distance-fare relationships, passenger patterns, weekday/weekend differences, and the comparison for 2024-2025. The year comparison is descriptive and observational; thus, differences in 2024 and 2025 are presented as associations and cannot be interpreted as evidence that congestion pricing brought about the observed change.

3.2 Platform Architecture

Layer / component	Actual implementation	Role in project
Official source	NYC TLC monthly Yellow Taxi Parquet + Taxi Zone Lookup	Public dataset acquisition
Amazon S3	Private raw/yellow and reference prefixes	Durable cloud source layer; Block Public Access and SSE-S3 enabled
EC2 master	NameNode, SecondaryNameNode, ResourceManager, Spark driver	Cluster coordination and spark-submit
EC2 worker	DataNode and NodeManager	HDFS block storage and YARN

Layer / component	Actual implementation	Role in project
		executor resources
HDFS	/data/nyc/raw, /processed/yellow_clean, /results	Distributed execution data layer
Spark / YARN	PySpark DataFrame API + Spark SQL	Profiling, preprocessing, aggregation and benchmark execution
GitHub	jie-long-spark branch	Reproducible code, small outputs, logs and evidence index

Table 3.2.1 Final platform architecture and component roles

The final architecture was based on the Hadoop/YARN model, with Amazon S3 serving as the entry point to the durable cloud storage. Raw monthly Parquet files and Taxi Zone Lookup were kept privately in an S3 bucket. The production pipeline used a two node EC2 Hadoop cluster, one master node for the NameNode, SecondaryNameNode and ResourceManager and one worker node for the DataNode and NodeManager, because the planned EMR cluster could not be provisioned successfully in the AWS Academy environment. Spark applications were submitted from the master in YARN client mode and read the central processed Parquet dataset from HDFS.

Architecture that is a separation of storage, computation and collaboration. The original cloud copy of the source files was held by S3, the staged raw data in HDFS, the central cleaned Parquet data and small directories containing results during execution, the PySpark DataFrames and Spark SQL transformations and aggregations, and only source code, configuration, documentation and small result artifacts were stored in GitHub. The raw 24-month data, credentials and private keys were not included on GitHub.

3.3 Data Acquisition and Storage Method

Dataset acquisition was controlled through a manifest before cloud processing. Exactly 24 monthly files named yellow_tripdata_YYYY-MM.parquet and one taxi_zone_lookup.csv were verified. The monthly files were uploaded to the private S3 raw/yellow prefix, while the lookup file was stored in the S3 reference prefix. The S3 bucket was configured with Block Public Access enabled and default SSE-S3 encryption. The raw layer has not been altered in subsequent processing.

The course AMI does not contain the AWS CLI, Hadoop S3A libraries or boto3.

Instead of installing additional packages, a Python 3 standard-library AWS Signature Version 4 transfer utility was used with temporary Learner Lab credentials to stage the verified S3 objects onto the master and then into HDFS. Exactly 24 Parquet files and the lookup were validated in HDFS, after which the temporary credentials were removed. This ensured that the private S3 source layer was not lost and the manual Hadoop cluster was able to process the dataset using HDFS.

3.4 Data Profiling and Preprocessing Method

One preprocessing contract was used for the Spark full-data pipeline. The method was designed to clearly distinguish between structural cleaning and validity for the analysis. Structural cleaning determined if a trip was stored in the central dataset, while valid_fare and valid_distance determined if a trip could be used for fare- or distance-based statistics. This distinction retained useful demand records even when their fare or distance values were unusual.

3.4.1 Raw Data Profiling

These monthly Parquet files were profiled on their own, prior to any filtering. For every month, Spark recorded raw row count, column count, a schema signature, minimum and maximum pickup/drop-off timestamps, and per-column null counts. By profiling month by month, the pressure on two t3.small nodes was eased, and the differences in schemas could be attributed to a particular source month. The lookup CSV was also validated on the number of rows, schema, distinct LocationID, and null and duplicate LocationIDs.

3.4.2 Schema Standardisation

The raw files were normalised to a common schema using explicit Spark casts. Timestamp columns were converted to timestamp; location and payment to integer-compatible; distance, fare, tip, total amount and fee to double. Case variants Airport_fee and airport_fee were normalised to airport_fee. When airport_fee or cbd_congestion_fee was absent, Spark created a typed null column. This enabled the 19-column 2024 schema and 20-column 2025 schema to generate one processed dataset which is compatible, while not assuming that an unavailable historical fee is zero.

3.4.3 Structural Cleaning

A row was considered structurally valid if both the pickup and the drop-off timestamps were not null, if the drop-off timestamp was after the pickup timestamp, if the computed duration was longer than 0 minutes and did not exceed 240 minutes, if the pickup year and month were the same as those in the source file's name, and if both PULocationID and DOLocationID were present. The 240-minute upper bound was a project-defined structural outlier rule. The count of issues per month has been determined prior to filtering; since there

are overlapping categories in the issues, the number of issues that were structurally removed from the data have been calculated as the number of raw rows minus the number of structurally valid rows.

3.4.4 Analysis-Specific Validity Flags

After the structural condition was defined, Spark created valid_distance = trip_distance > 0 and valid_fare = fare_amount > 0. Rows that did not pass these flags were retained in the structurally valid data portion of the central data and were used in trip-count analyses, but not in the aggregation of the relevant distance or fare averages. This allowed an otherwise valid trip record not to be invalidated by a non-positive fare or distance.

3.4.5 Feature Engineering

Temporal features were derived from the pickup_timestamp: pickup_year, pickup_month, pickup_date, pickup_hour, weekend, where Saturday and Sunday were classified as weekend days. trip_duration_min was calculated as drop-off_timestamp minus pickup_timestamp, in minutes. Distance was grouped into 0–<2, 2–<5, 5–<10, 10–<20 and 20+ mile bands, with invalid or non-positive values assigned to a separate category.

Rule area	Final Spark rule
Structural duration	0 < trip_duration_min <= 240
Source-month check	pickup_year/month must equal the source filename period
Location IDs	PULocationID and DOLocationID must be non-null
valid_distance	trip_distance > 0
valid_fare	fare_amount > 0
Distance bands	0-<2; 2-<5; 5-<10; 10-<20; 20+; Invalid/Non-positive
Passenger groups	Missing; 0; 1-6; 7+
Lookup	Left join PULocationID -> LocationID; preserve unmatched rows and count them

Table 3.4.1 Final preprocessing and feature-engineering rules

3.4.6 Taxi Zone Enrichment

The Taxi Zone Lookup was cast once, renamed as pickup_Borough, pickup_Zone and pickup_service_zone prior to performing a left join between PULocationID and LocationID. The approved analysis scope only demanded the pickup join. A left join was performed to prevent the loss of a structurally valid trip due to the absence of its lookup value. A zero number of unmatched PickupLookupRows were created in the final run.

3.4.7 Processed Data Storage

Each successful monthly clean DataFrame was written as Parquet to a month-specific HDFS staging path. Once all 24 months had been reached, the month directories were transferred to the final logical dataset /data/nyc/processed/yellow_clean, which was split into two partitions: pickup_year and pickup_month. Multiple Parquet part files were intentionally preserved; coalesce(1) was not used on the full dataset. The completed structure had 24 month partitions, and 2-year partitions.

3.4.8 Centralised Dataset Consistency

All full-period Spark analyses will then read the same central processed path, instead of reading the raw monthly files and reprocessing through the Spark again. This ensured that each month's demand, hourly demand, location, distance, passenger outputs, and payment outputs were based on a common structural baseline of 89,291,810 records. Small CSV summaries were exported separately for validation, visualisation and handover to my partner.

3.5 Spark SQL and PySpark Analytical Method

The analytical stage used the PySpark DataFrame API as the main programmatic interface and Spark SQL as a second expression of selected aggregations. The cleaned Parquet data set was loaded in one go and stored as a temporary view called yellow_taxi. DataFrame aggregations produced seven compact analytical outputs: monthly trip demand, hourly weekday/weekend demand, pickup borough summary, top 20 pickup zones, distance-band summary, passenger-group summary and payment-type summary. All trip-count analyses were performed with the entire valid dataset, and conditional expressions only applied to averages where they were needed.

Spark was used as the comparable distributed-processing approach to the traditional MapReduce model. Row-level transformations such as filtering and feature derivation operate independently across data partitions, while operations such as joins, groupBy and aggregation require data to be exchanged across partitions before grouped results are reduced. Spark manages these operations through its execution plan and DAG rather than requiring separate Map and Reduce programs. The partition-filter and Exchange evidence presented in Section 4.6 demonstrates these distributed processing stages in the implemented workload.

Four analyses were repeated in Spark SQL: monthly demand, hourly weekday/weekend demand, borough summary and distance-band summary. All counts had to be equal and floating-point averages were compared by value. All four DataFrame-versus-SQL comparisons in the resulting validation file were PASS. This gave an internal

correctness check of the results expected to be consistent across both interfaces of Spark.

ID	Analysis	Main operation
A1	Monthly trip demand	pickup_year, pickup_month -> trip_count
A2	Hourly weekday/weekend demand	weekend, pickup_hour -> trip_count
A3	Pickup borough summary	trip_count, conditional avg distance/fare, avg duration
A4	Top pickup zones	pickup_Borough + pickup_Zone ranked by trip_count
A5	Distance-band summary	trip_count, avg duration, conditional avg fare
A6	Passenger-group summary	trip_count, conditional avg distance/fare
A7	Payment-type summary	trip_count and conditional fare/tip/total averages
SQL1-SQL4	SQL equivalents	A1, A2, A3 and A5 repeated through yellow_taxi view

Table 3.5.1 Spark DataFrame and Spark SQL analytical plan

3.6 pandas Analytical Method

This was implemented in pandas as an independent local implementation for the entire 24-month period from January 2024 to December 2025. Each official Yellow Taxi Parquet file was read with the PyArrow engine and processed one month at a time. This sequential approach did not concatenate around 89 million records into one in-memory DataFrame, but still took the same rules to apply the same timestamp, location, source-month and 240-minute maximum-duration logic as in the Spark workflow.

The program was able to obtain pickup year, month, date, hour, weekend status and trip duration for each month of the file. Records with non-positive distances or fares were still included in the records, but the presence of valid_distance and valid_fare flags determined which statistics those records could contribute to. The official Taxi Zone Lookup was joined to pickup location identifiers using a many-to-one left merge. Compact cumulative aggregates were preserved for distance-fare, passenger-count, weekday/weekend and year-comparison analyses.

For section 6.1, the pandas workflow chose January 2024 as a common correctness-validation period to ensure the same source month, same pre-processing rules and Taxi Zone

Lookup is compared with Spark. Exact agreement was required for the structurally valid row count and leading pickup zone. The full-period pandas outputs were retained for the broader analytical sections, while the separate 1, 3, 6, 12 and 24-month workloads were used for scalability benchmarking.

3.7 Analytical Plan

The analytical plan linked each research question to a saved aggregate rather than relying on ad-hoc notebook output. Temporal demand was expressed as monthly and hourly counts, location concentration through borough and zone rankings, trip characteristics through distance and passenger group, and Spark implementation behaviour through Dataframe validation and physical plan inspection. A detailed fare-per-mile and passenger-count visualisation, as well as a visualisation of 2024-versus-2025, were prepared by pandas/visualisation, with shared full-period definitions.

Analytical area	Saved evidence / output
Dataset quality	profile_summary.txt; monthly_profile.csv; monthly_quality.csv
Temporal demand	monthly_trip_demand.csv; hourly_weekend_demand.csv
Pickup locations	pickup_borough_summary.csv; top20_pickup_zones.csv
Distance/fare	distance_band_summary.csv plus full-period visualisation summaries
Passenger patterns	passenger_group_summary.csv plus detailed pandas visualisation summaries
Payment behaviour	payment_type_summary.csv
Spark SQL consistency	dataframe_sql_validation.csv
Spark performance	benchmark_results.csv and four formatted physical-plan files

Table 3.7.1 Analytical areas and saved outputs

3.8 Validation and Benchmark Design

Correctness validation is done on the basis of January 2024 as a common validation month. Both implementations read the same Yellow Taxi Parquet data from January 2024 and implemented the same structural-validity rules: valid pickup and drop-off timestamps, source-month consistency, non-null pickup and drop-off location identifiers, and trip duration greater than 0 and no more than 240 minutes. The same Taxi Zone Lookup was used for pickup-location enrichment. This allowed for a comparison of correctness with a similar input domain and analytic definition.

Structurally valid row count and top pickup zone had to agree for the correctness checks to be

cross-platform. The January pandas baseline also included the busiest pickup hour and average valid fare by distance band, but these are not shown as cross-platform PASS checks because an equivalent January-specific Spark output was not saved. This January correctness validation is kept separate from the 1, 3, 6, 12 and 24-month matched scalability benchmark.

Scalability is calculated based on a matched top-pickup-zone workload at 1 month, 3 months, 6 months, 12 months and 24 months. Both implementations begin with the same corresponding raw monthly Parquet files, follow the same rules for structural-validity rules for timestamps, source-month consistency and location identifiers, and then aggregate the number of valid trips by pickup location, add the Taxi Zone Lookup to the ranking, and return the winning pick-up zone. SparkSession startup and lookup preparation are excluded from the Spark timing, while the timer ends only after the final Spark action materialises the ranked result. Spark is executed three times per scale where feasible and the median runtime is used for comparison. The comparison focuses on the growth of runtime and feasibility of completion and scalability, as pandas and Spark are executed on different hardware and different execution architectures.

The benchmark interpretation is based on runtime growth, completion feasibility, behaviour of memory use and platform properties. Spark is not assumed to be faster for every workload; local pandas can have lower scheduling overhead for compact sequential aggregations, while Spark provides distributed execution, partition pruning and scalable cloud processing for the full data pipeline.

3.9 Visualisation Method

Matplotlib is used to convert the aggregated analytical outputs into report-ready figures. Appropriate chart types are selected according to the analytical purpose: line charts are used for hourly and monthly trends, bar charts for categorical comparisons such as distance bands and passenger groups, and horizontal bar charts for ranked location results. All figures are generated from summarised data that improves readability and reduces unnecessary local memory usage. The final figures support interpretation of hourly demand, pickup-zone concentration, distance and fare patterns, passenger behaviour, weekday versus weekend differences, 2024 versus 2025 trends, correctness validation and runtime comparison.

4.0 Implementation

4.1 GitHub Repository and Collaboration

A shared GitHub repository, ItzSeir/Small-Data-Analytic, was used to manage the project source code, configuration files, documentation and selected result outputs. To maintain the structure of this project, two separate development branches were created for the

Spark and pandas implementations. The repository includes pre-processing rules, dataset information, analysis scripts, results from the benchmarks, documentation of execution and supporting evidence necessary for the reproducibility.

Only lightweight and reproducible project artifacts were stored in GitHub. Large raw and processed datasets, cloud credentials, session tokens, private key files and large runtime logs were excluded from the repository. This way the repository remains organized, safe and works for collaboration without unnecessary duplication of the official public data set.

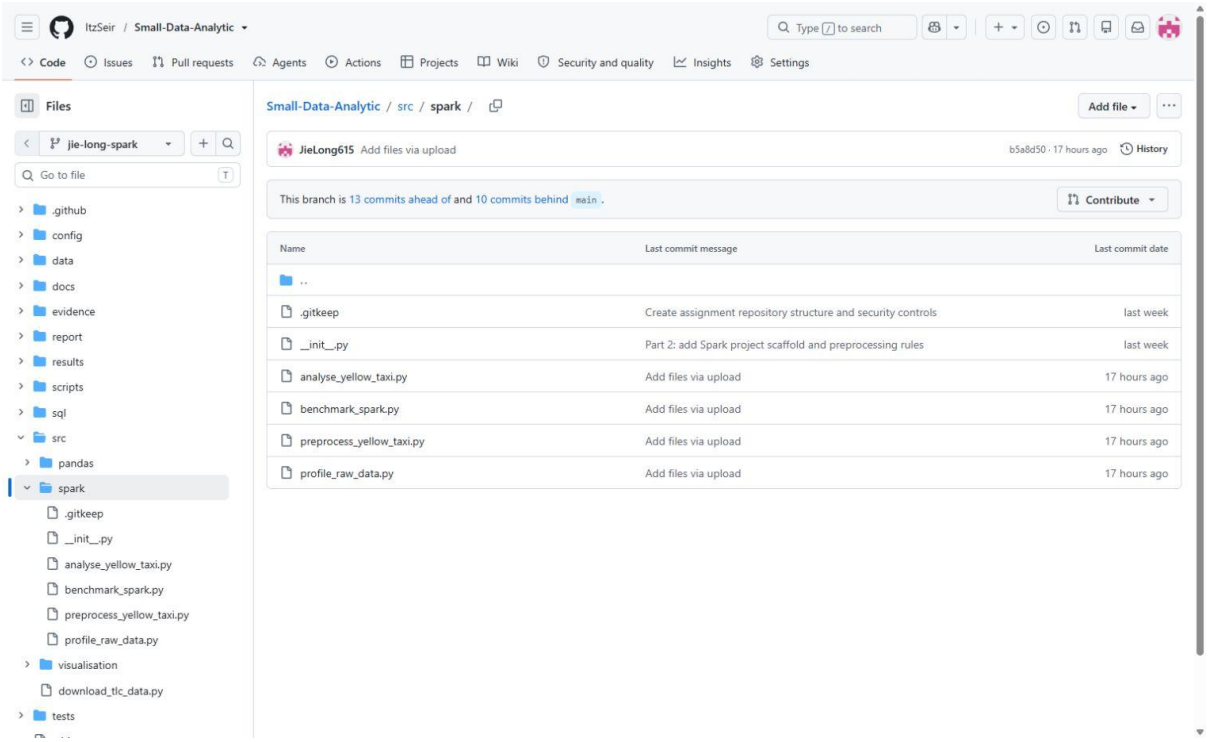


Figure 4.1.1 Final PySpark source scripts available on the jie-long-spark branch.

4.2 Amazon S3 Organisation

Amazon S3 was used as the source layer in the private cloud. All 24 monthly Parquet files were in the nyc-taxi/raw/yellow/ prefix and taxi_zone_lookup.csv was in the nyc-taxi/reference/ prefix. The bucket leveraged S3 Block Public Access and default SSE-S3 encryption. Profile, preprocessing, analysis and benchmark artifacts were also added as dedicated result prefixes, but the central dataset was actually processed in HDFS on the self-managed EC2 Hadoop/YARN/Spark cluster.

Data staging required an implementation adjustment. The course Hadoop AMI did not contain AWS CLI, S3A connector JARs and boto3. A Python 3 standard library SigV4 downloader thus authenticated with temporary credentials from the Learner Lab, verified 24 unique months, downloaded the source files, copied them to HDFS and deleted the temporary

credentials. The same SigV4 technique was later used to export a small GitHub artifact bundle.

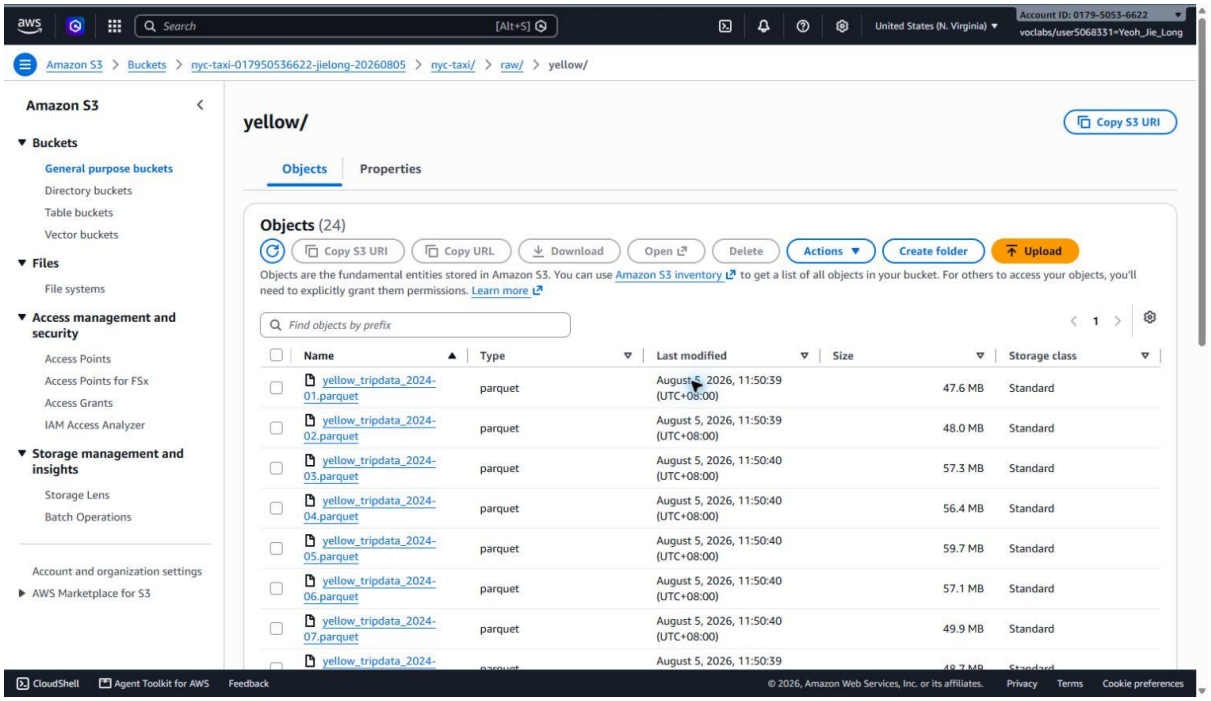


Figure 4.2.1 Private S3 raw layer containing the 24 Yellow Taxi monthly Parquet files.

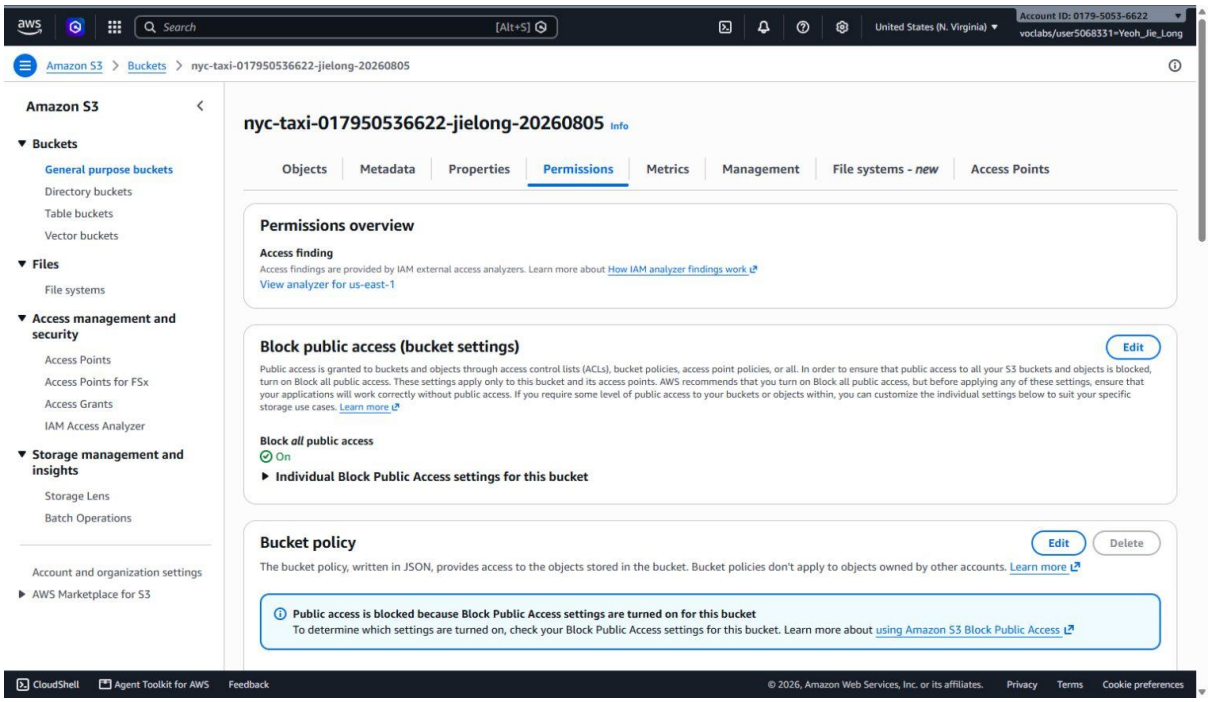


Figure 4.2.2 S3 Block Public Access configuration for the project bucket.

4.3 AWS Cluster Configuration and EC2 Hadoop/YARN Implementation

The original design was based on the managed-service design pattern used in the course EMR lab. Cluster ist3134-nyc-taxi-jielong (ID j-2O93TR9KEVKN6) was created with EMR 7.13.0 and Spark/Hadoop, but the cluster has been stuck in the Starting state for a while with multiple instance attempts getting terminated, finally reaching the TERMINATED_WITH_ERRORS state with INTERNAL_ERROR. CloudTrail evidence showed the Academy/Vocareum environment terminating the attempted nodes. The project didn't declare itself the successful production platform for the reason that it was an infrastructure failure and there was no stable EMR cluster available.

The final architecture thus adapted the manual Hadoop and YARN approach taught in IST3134 Labs 2 and 4. Two existing EC2 instances were configured as a master and worker in the same VPC/security group. The initial t3.micro configuration proved to be memory insufficient for the desired Spark configuration, and the two existing instances were stopped, resized to t3.small, and restarted without rebuilding the cluster. The master ran NameNode, SecondaryNameNode and ResourceManager, while the worker ran DataNode and NodeManager. HDFS reported one live DataNode and YARN one running node. Spark-on-YARN smoke test returned spark.sparkContext.master = yarn and a count of 1000, which meant it was confirmed that Spark jobs could be scheduled through the cluster.

Setting	Final configuration / validation
AWS region	us-east-1
Master	i-0130507b6de6a79a3; t3.small; 30 GiB gp3
Worker	i-048b900fcc5a38f15; t3.small; 30 GiB gp3
Master services	NameNode, SecondaryNameNode, ResourceManager
Worker services	DataNode, NodeManager
HDFS	fs.defaultFS = hdfs://master:9000; 1 live DataNode
YARN	1 running node
Spark execution	YARN client mode; final heavy jobs used 1 executor, 1 core, 768 MB executor memory and 768 MB driver memory
Smoke validation	HDFS read succeeded; Spark master = yarn; Spark count = 1000

Table 4.3.1 Final EC2 Hadoop/HDFS/YARN/Spark cluster configuration

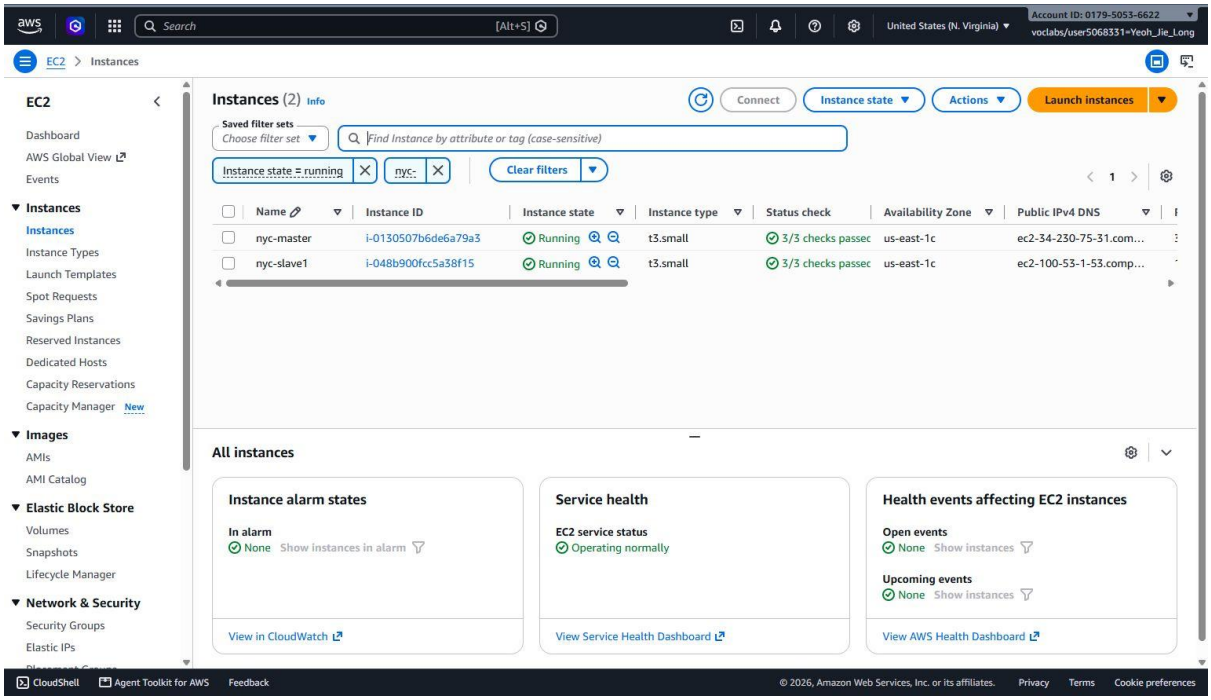


Figure 4.3.1 Final two-node t3.small EC2 cluster used for the successful Spark implementation.

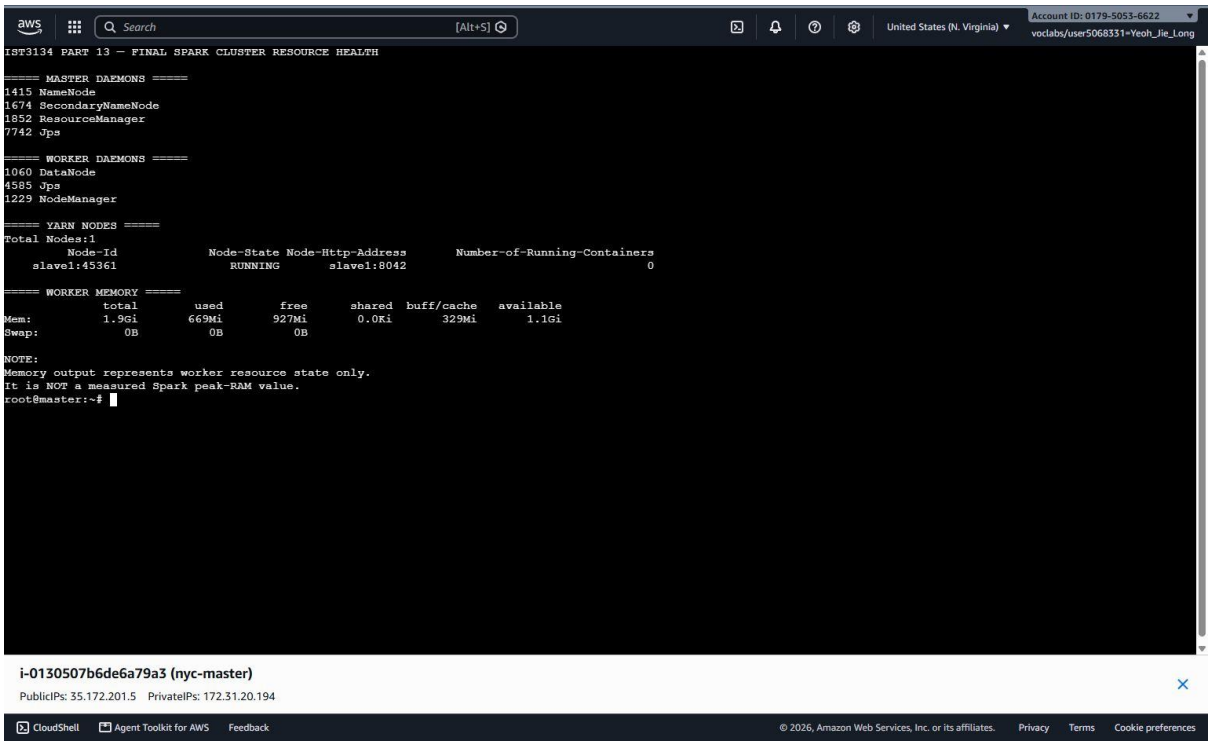


Figure 4.3.2 Final Hadoop/YARN cluster health showing the master and worker daemons and one running YARN node

4.4 PySpark Preprocessing and Centralised Dataset Implementation

The production preprocessing script used was src/spark/preprocess_yellow_taxi.py. It processed the 24 files sequentially rather than combining all raw months in one in-memory DataFrame. For each source month, Spark normalized the schema, added time/duration data, computed quality diagnostics, applied structural validity, generated analysis flags/categories, added pickup-zone attributes, and loaded out the valid rows into a month-specific HDFS staging directory. Only after every month completed successfully were the staging partitions moved into the final /data/nyc/processed/yellow_clean dataset.

Across 89,892,322 raw rows, the structural rules retained 89,291,810 rows and removed 600,512. There were a total of 2,179,263 raw rows of non-positive/null distance diagnostics and 3,618,518 non-positive/null fare diagnostics, but these were not used as structural removals. Valid_distance and valid_fare, therefore, only excluded invalid values when they were necessary for later analysis. The pickup-location enrichment produced zero unmatched lookup rows. The final Parquet layout contained 24 monthly partitions in total, with 12 partitions under 2024 and 12 under 2025 of the pickup_year column. HDFS reported approximately 2.1 GB logical size and 4.2 GB storage consumption for the final clean path at evidence capture.

An executor heartbeat timeout occurred while writing the December partition. No manual data correction needed: YARN replaced executor and Spark completed the retried task. The final clean row count was re-read from the Parquet output and equaled the structurally valid total of 89,291,810, a complete validation of the write phase.

Preprocessing outcome	Result
Raw rows	89,892,322
Structurally valid	89,291,810
Structurally removed	600,512
Raw non-positive/null distance diagnostic	2,179,263
Raw non-positive/null fare diagnostic	3,618,518
Unmatched pickup lookup	0
Final year/month partitions	2 years / 24 months
Final HDFS path	/data/nyc/processed/yellow_clean

Table 4.4.1 Spark preprocessing outcome summary

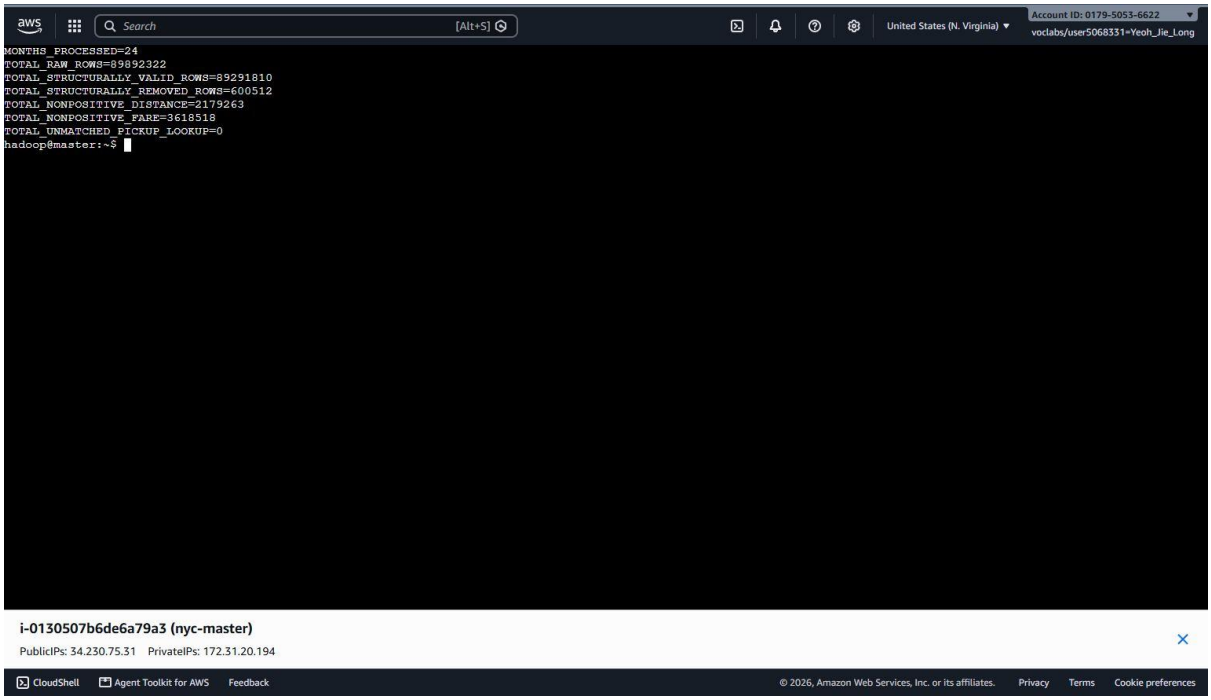


Figure 4.4.1 Spark preprocessing totals for all 24 monthly files.

4.5 Spark SQL Analysis Implementation

The full analytical script was src/spark/analyse_yellow_taxi.py. It loaded in /data/nyc/processed/yellow_clean, created a temporary view named yellow_taxi, and produced seven tables in DataFrame. All rows that were structurally valid were used in the trip count. The distance and fare averages were based on conditional expressions, meaning that rows that were marked invalid for the distance or fare measure did not affect the corresponding statistic. Result files were aggregated only after being written, while small CSV summaries were written locally and to HDFS, holding the large trip-level data set distributed.

Spark SQL ran four representative aggregations on the yellow_taxi view. The DataFrame and SQL outputs were compared by grouping keys, exact trip counts and numeric averages. The global count of each of the four comparisons was each successful, and the totals for each of the distance-bands and passenger-groups each totalled 89,291,810. This validated both the central processed dataset and the consistency of the two Spark interfaces.

```
# Representative Spark SQL aggregation used in the project
SELECT pickup_year, pickup_month, COUNT(*) AS trip_count
FROM yellow_taxi
GROUP BY pickup_year, pickup_month;
```

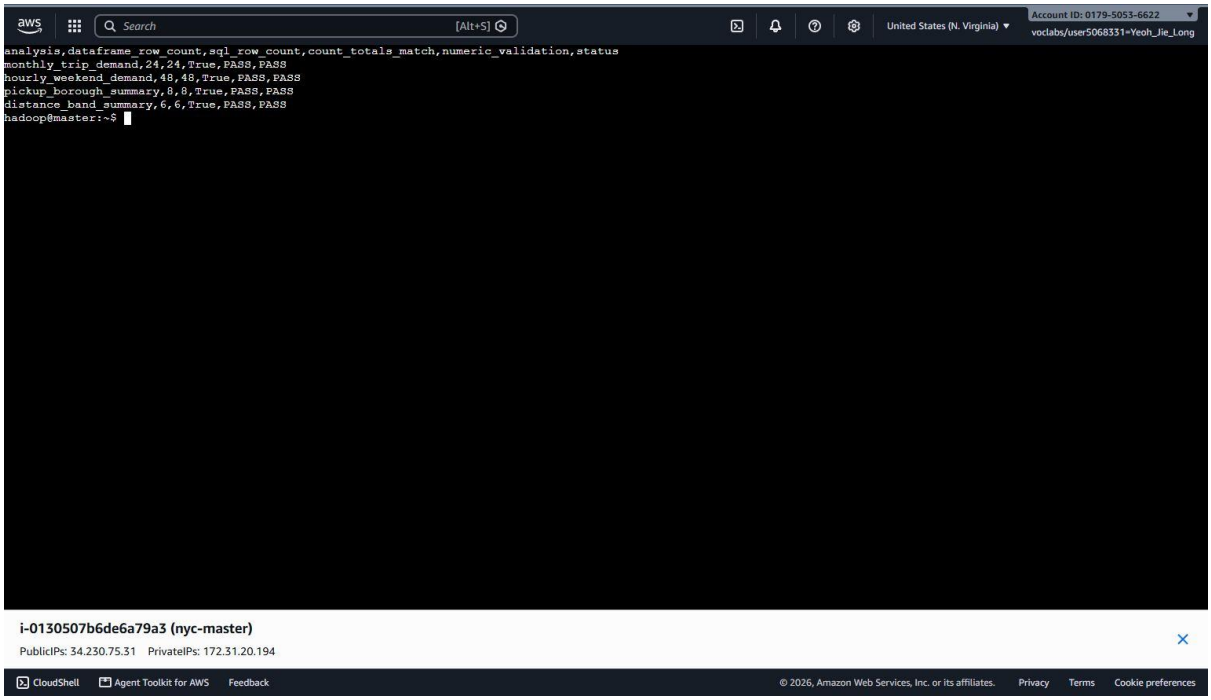


Figure 4.5.1 DataFrame-versus-Spark-SQL validation with all four analytical checks passing.

4.6 Spark Performance and Physical Plan Implementation

The performance evidence was generated by src/spark/benchmark_spark.py. The benchmark was kept lightweight because of the limited resources of the two-node Learner Lab cluster. It ran 2 partition-count operations and aggregated a monthly-demand over a period of 24 months, with and without the DataFrame API and Spark SQL. Each saved timing represents one observed run on the final cluster and is therefore reported descriptively rather than as a universal performance claim.

Benchmark	Method	Rows	Elapsed (s)	Validation
January 2024 partition count	DataFrame count	2,961,858	3.125253	PASS
Full 2024 partition count	DataFrame count	41,132,445	1.108304	PASS
Monthly demand - full clean dataset	DataFrame API	89,291,810	5.322686	PASS
Monthly demand - full clean dataset	Spark SQL	89,291,810	3.345567	PASS

Table 4.6.1 Initial Spark benchmark results and validation

Formatted physical plans were saved for all benchmark queries. In January 2024, the plan included partition filters for pickup_year=2024 and pickup_month=1, and in the full-2024 plan, there were only partition filters for pickup_year=2024. The monthly aggregation plans had Exchange operations, which is a direct proof of shuffle in GroupBy processing. The fact that the larger count was completed in 2024, albeit earlier than the January count, is why a

single wall-clock comparison should not be interpreted as a controlled scaling result; startup, caching, scheduling and storage effects can have a significant impact in short runs.

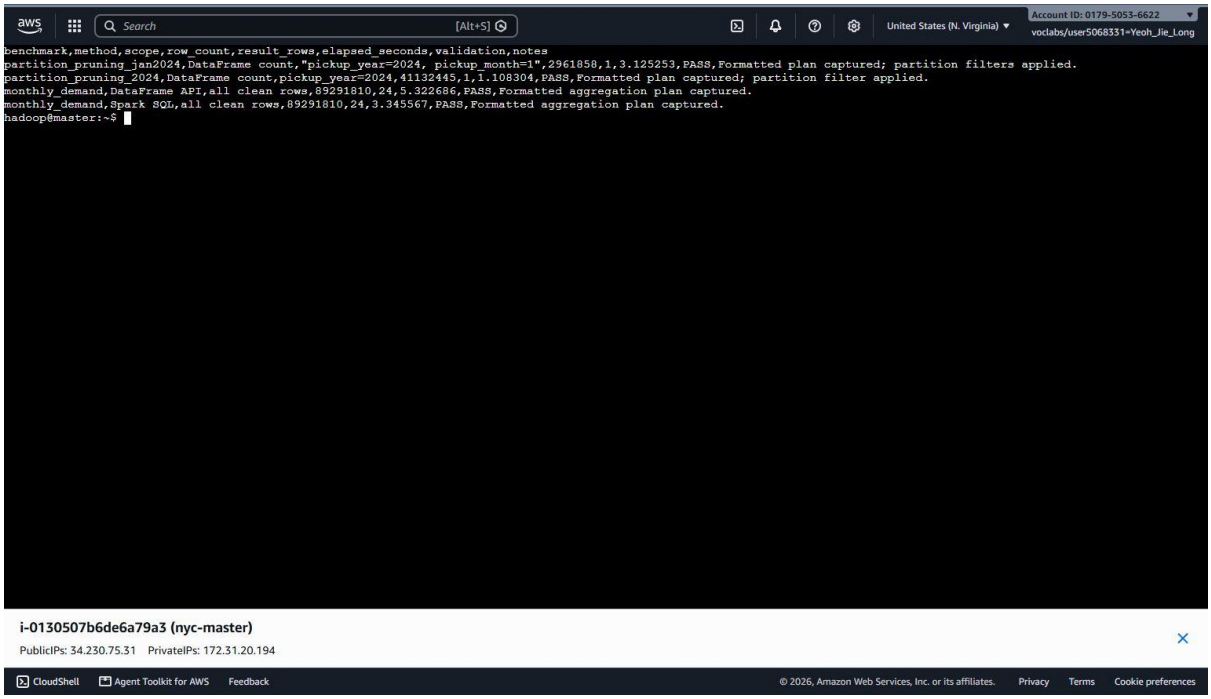


Figure 4.6.1 Observed Spark benchmark runtimes on the final two-node cluster.

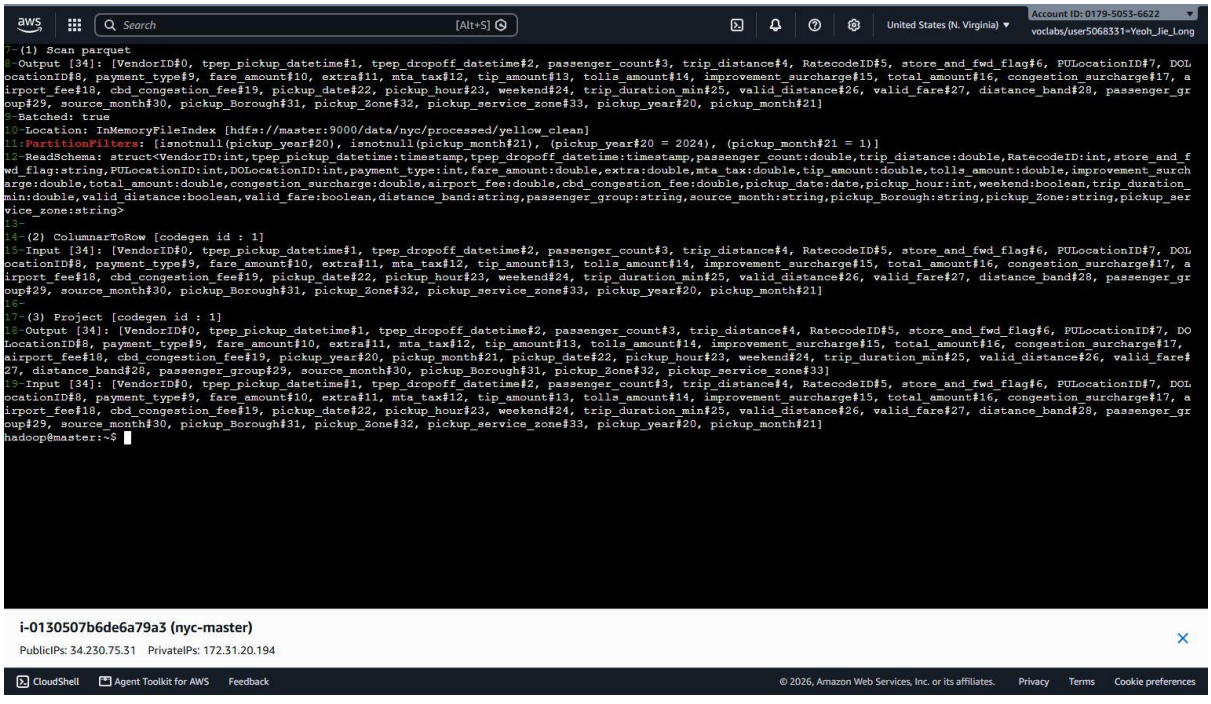


Figure 4.6.2 Spark physical-plan excerpt showing partition filters for January 2024.

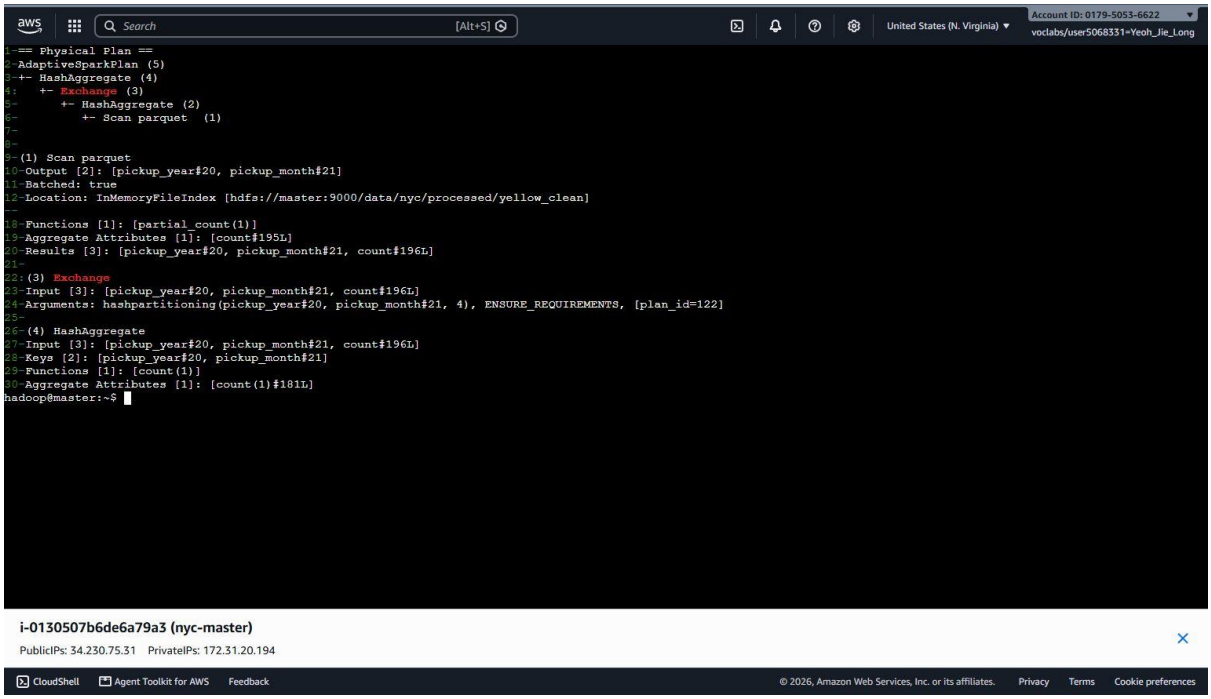


Figure 4.6.3 Spark physical-plan excerpt showing Exchange/shuffle in the monthly aggregation.

4.6.1 Matched Spark-pandas Scalability Benchmark

Following the above Spark-only physical-plan benchmark, another matched benchmark was performed specifically for comparison between Spark and pandas. The workload reads the same structure-validity rules from the same monthly Parquet file on HDFS, counts the number of rows that are structure-valid, aggregates pickup locations, joins the Taxi Zone Lookup, and returns the top ranked Taxi Zone. The preparation of startup and lookup of SparkSession was not included in the timed part, and the timing did not start until the ranked Spark result was materialised. The benchmark used the same five scales as pandas: 1, 3, 6, 12 and 24 months. All scales began in January 2024 with the 24-month scale concluding in December 2025.

The last two-node EC2 cluster with one t3.small master and one t3.small worker was used for the matched Spark runs under HDFS/YARN. The benchmark constrained Spark to one executor, one executor core, 512 MB executor memory, 256 MB executor overhead, and 768 MB driver memory. Three runs of Spark were kept at each scale, and the median of these runs was reported as the runtime. All five scales matched panda exactly for the structurally valid rows, and top pickup zone and top-zone trip count.

Months	Valid rows	Spark median (s)	Top pickup zone	Top-zone trips	Validation
1	2,961,858	15.634926	JFK Airport	145,025	PASS

Months	Valid rows	Spark median (s)	Top pickup zone	Top-zone trips	Validation
3	9,546,211	28.577788	Midtown Center	453,523	PASS
6	20,313,815	48.814859	Midtown Center	954,242	PASS
12	41,132,445	93.906438	JFK Airport	1,987,052	PASS
24	89,291,810	191.775759	JFK Airport	4,034,602	PASS

Table 4.6.2 Matched Spark top-pickup-zone benchmark results for 1, 3, 6, 12 and 24 months.

A 24-month observation of 666.948375 seconds was removed from the performance summary due to the executor exiting with a code of 137 during that observation. YARN recovered and the analytical result remained correct, so the event was retained as fault-recovery evidence rather than treated as a clean timing. A clean observation of 211.714533 seconds was then taken and a 24-month median was formed using three clean observations, which resulted in a final 24-month median of 191.775759 seconds.

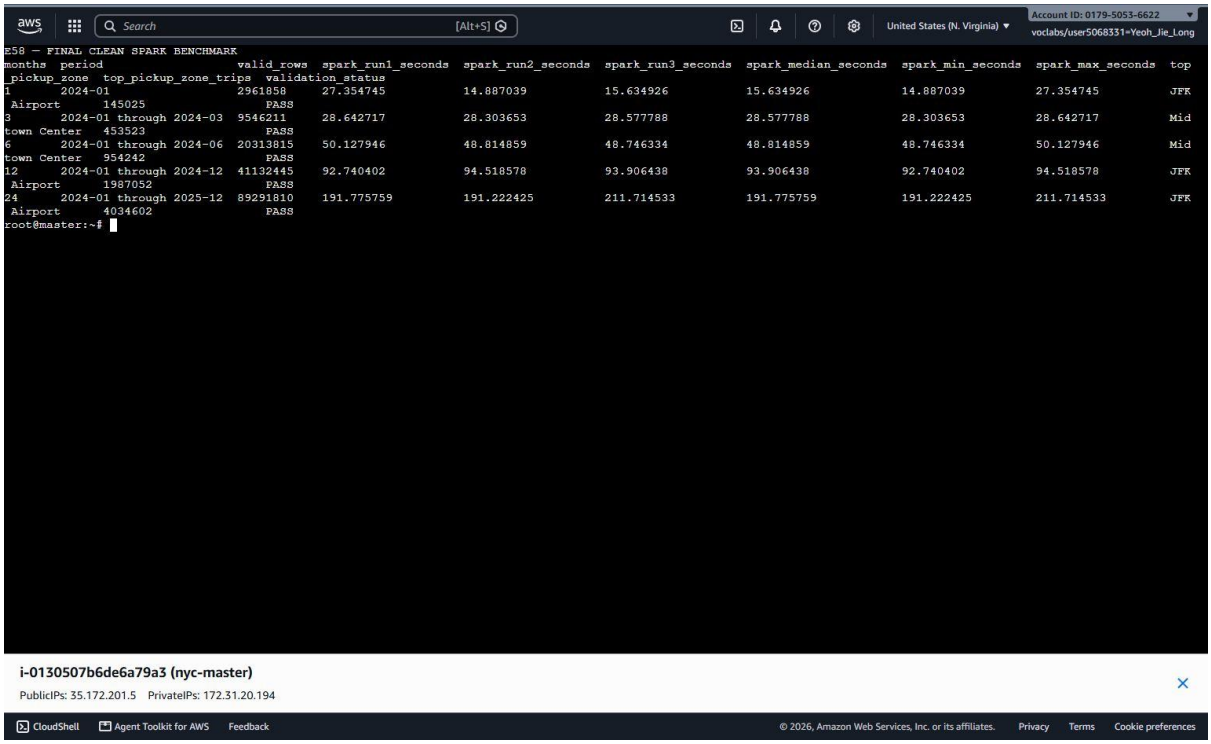


Figure 4.6.4 Final clean Spark benchmark showing all five matched scales, three clean Spark observations per scale, median runtime and exact PASS validation.



Figure 4.6.5 Clean 24-month Spark replacement run collected after excluding the executor-exit-137 observation from the performance summary.

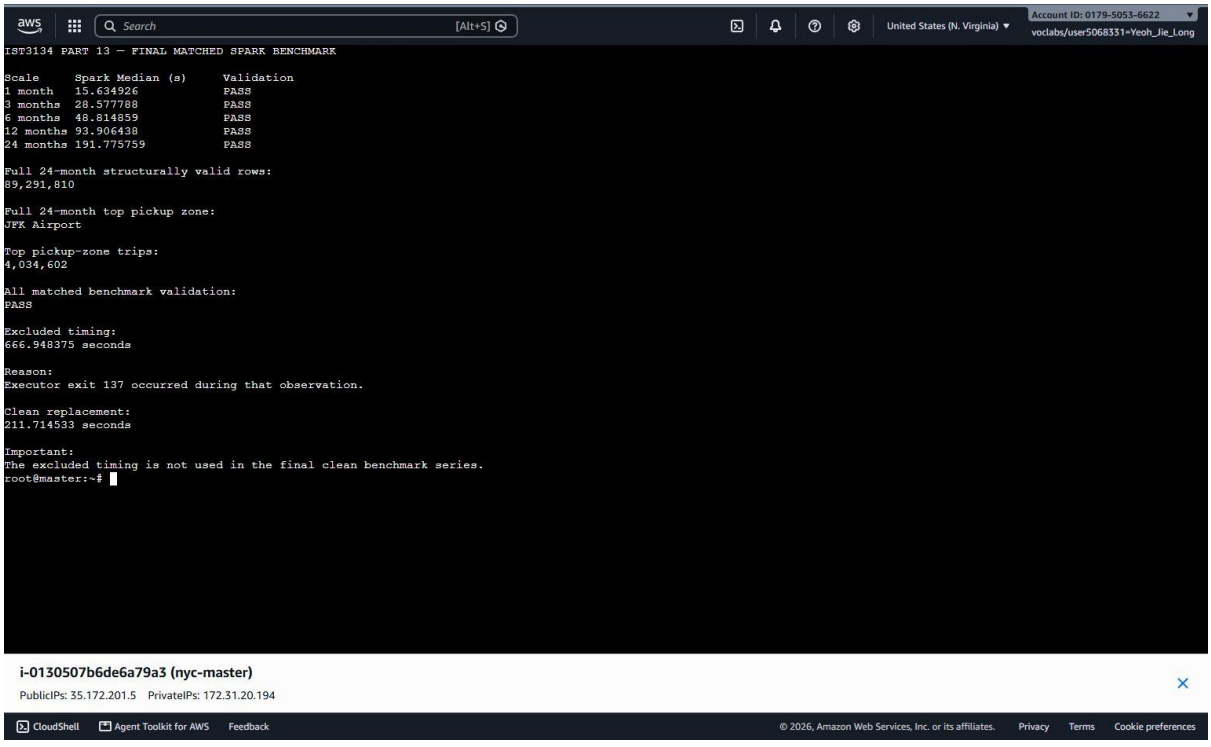


Figure 4.6.6 Final Spark median-runtime and validation summary used for the matched Spark-versus-pandas comparison.

4.7 pandas Preprocessing and Analysis Implementation

The final pandas implementation was performed, as a full-period local workflow, on all 24 Yellow Taxi Parquet files that span January 2024 to December 2025. To control local

memory use, the program processed one monthly file at a time and retained only compact cumulative aggregates rather than concatenating all 89,892,322 raw records into a single DataFrame. Each month was standardised, structurally cleaned and enriched using the Taxi Zone Lookup before its aggregated results were merged into the full-period summaries.

The entire pandas analysis took 89,892,322 raw records in and ended up with 89,291,810 structurally valid records out. It found that 18:00 is the most congested hour for pickup (6,384,616), and that JFK Airport is the busiest pickup zone (4,034,602).

The implementation generated common metrics, monthly quality results, distance-fare summaries, detailed passenger-count outputs, weekday/weekend summaries and 2024-versus-2025 monthly summaries. The scalability benchmark was also extended to the complete 24-month period. All five scales tried were successfully completed, which demonstrated that the sequential aggregation design was still viable on the local machine.

Months	Valid rows	Runtime (s)	Peak observed RSS (MB)	Status
1	2,961,858	0.464	465.67	SUCCESS
3	9,546,211	1.368	485.34	SUCCESS
6	20,313,815	3.040	487.79	SUCCESS
12	41,132,445	5.937	489.34	SUCCESS
24	89,291,810	12.814	513.42	SUCCESS

Table 4.7.1 pandas benchmark execution results

4.8 Visualisation Implementation

The final visualisation workflow used the compact CSV summaries produced by the full-period pandas analysis. Matplotlib generated report-ready figures without reloading the complete 24-month raw dataset. This separation between aggregation and visualisation improves reproducibility and prevents chart generation from becoming another high-memory operation.

The updated figures cover average and median fare, average total amount, fare per mile and duration by distance band; passenger-count distribution and single-versus-multiple passenger shares; weekday-versus-weekend hourly demand; monthly trip volume, average daily trips, average fare and CBD-fee charged share for the 2024-2025 comparison; and pandas runtime and memory growth. All figure titles identify the analytical measure and period, and all values are traceable to saved CSV outputs.

4.9 Testing and Reproducibility

Testing was performed at several levels. The repository contained boundary tests and output-schema tests for the shared preprocessing rules, and all seven tests passed before AWS execution. Cluster smoke tests were performed to verify HDFS read/writes behaviour, 1 live DataNode, 1 running YARN node and 1 running Spark-on-YARN execution with 1000 count. During profiling and preprocessing, month counts and output line counts were validated after each stage. The clean Parquet dataset was then read in the final time and counted and found to have 89,291,810 rows, which matched the structural-validity count exactly.

Analytical reproducibility was checked by comparing DataFrame and Spark SQL outputs for four aggregations, all of which passed. Repository controls also supported reproducibility: source scripts, configuration, small CSV outputs, benchmark plans, execution log and evidence index were committed to jie-long-spark, while .gitignore and manual checks excluded raw Parquet files, credentials and key material. The assignment therefore retains both executable code and traceable result artifacts rather than relying on screenshots alone, consistent with the submission requirement.

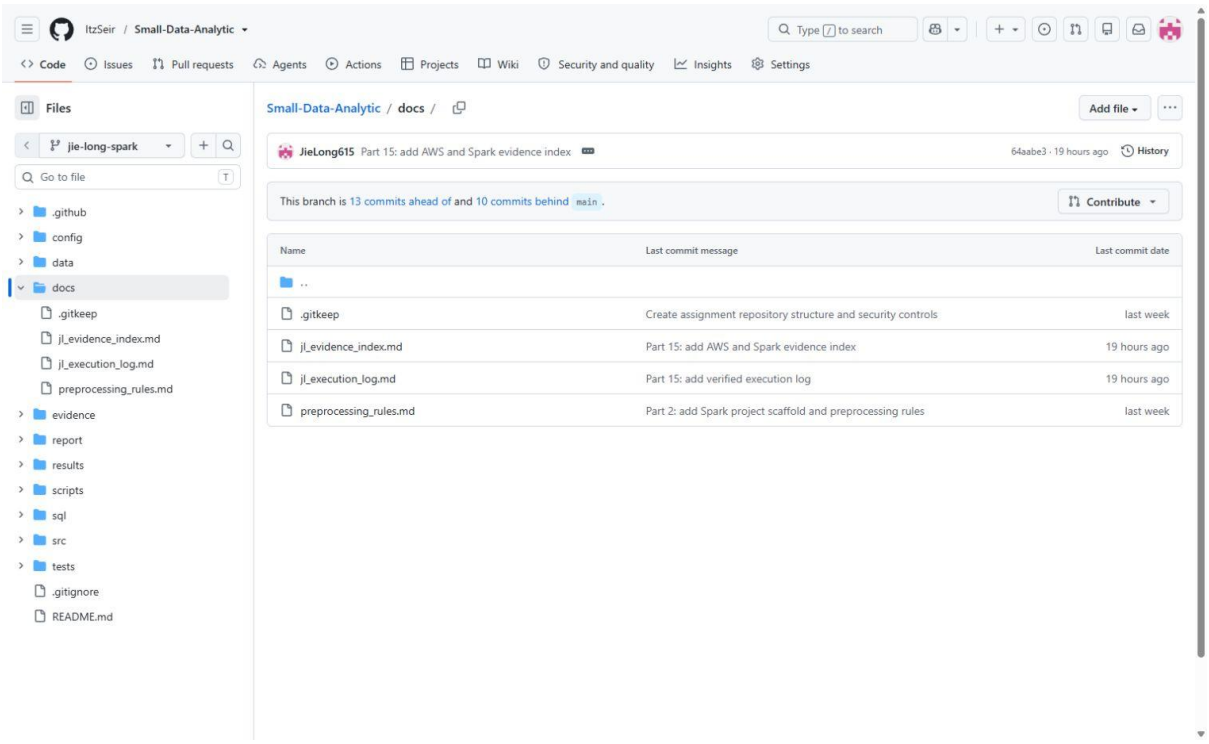


Figure 4.9.1 GitHub documentation containing the JL execution log, evidence index and preprocessing rules.

Before the final matched scalability benchmark, the existing EC2 Hadoop/YARN environment was revalidated after the instance restart. Only the existing HDFS and YARN services were restarted; the NameNode was not reformatted and no data or Hadoop/Spark

configuration was changed. The pre-check revealed 1 live DataNode, 1 YARN NodeManager, 24 raw monthly Parquet files (2024-01 to 2025-12), Taxi Zone Lookup, about 17 GB of local free disk space, and 14.9 GB of free disk space in HDFS. HDFS had 117 under-replicated blocks and 0 missing and 0 corrupt blocks reported, and therefore the benchmark inputs were not lost or corrupted.

Matched benchmark then did exact validation at all scales. Three clean Spark observations were kept per scale, medians were taken, and executor-exit-137 observation was explicitly excluded from performance timing, but kept as operational fault-recovery evidence. This distinction allows for a reproducible comparison of the performance without concealing the failure event.

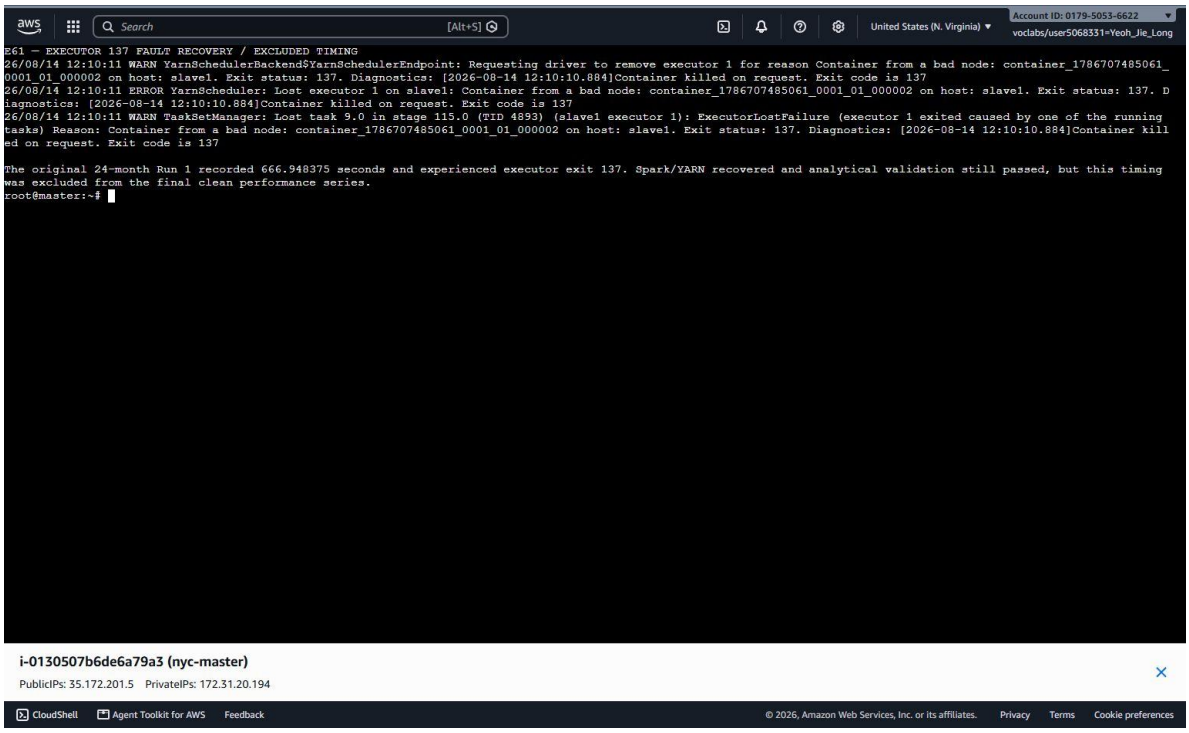


Figure 4.9.2 Executor-exit-137 fault-recovery evidence retained while the affected 666.948375-second observation was excluded from performance timing.

5.0 Result and Analysis

5.1 Dataset Volume and Raw Quality

The Spark raw profile processed all 24 monthly source files and counted 89,892,322 records prior to structural filtering. Raw volume increased from 41,169,720 records in 2024 to 48,722,602 in 2025. The highest number of raw files in any month was 4,591,845 in May 2025, while January 2024 had the lowest with 2,964,624. The 2024 files always included 19 columns and the 2025 files included 20 columns, with cbd_congestion_fee as the extra column.

The profile also illustrated the need for source validation. Some raw timestamps were historical (for example 2002, 2007, 2008 or 2009) and a small number extended far beyond the expected source month. Quality diagnostics identified 0 rows with a null drop off timestamp, 0 rows with a null pickup timestamp, 559,814 rows where the duration's drop off was not after the pickup, 40,094 durations greater than 240 minutes, and 634 rows of mismatch for the source month. Missing passenger_count was much more common, affecting 15,703,126 raw rows. Non-positive/null distance diagnostics failed 2,179,263 raw rows and non-positive/null fare diagnostics failed 3,618,518 raw rows.

Profile / quality item	Count	Interpretation
Raw rows	89,892,322	Total across 24 months
2024 raw rows	41,169,720	12 monthly files
2025 raw rows	48,722,602	12 monthly files
Drop-off not after pickup	559,814	Structural issue; overlaps with duration_nonpositive
Duration > 240 min	40,094	Structural outlier diagnostic
Source-month mismatch	634	Pickup month/year did not match filename
Missing passenger count	15,703,126	Retained structurally; handled in passenger grouping
Null/non-positive distance	2,179,263	Diagnostic before filtering; not a unique removal count
Null/non-positive fare	3,618,518	Diagnostic before filtering; not a unique removal count

Table 5.1.1 Raw dataset profile and quality diagnostics

5.2 Preprocessing Outcomes

Structural preprocessing retained 89,291,810 of 89,892,322 raw rows, equivalent to 99.332% of the source records, and removed 600,512 rows (0.668%). The removals of 600,512 were computed directly as rows that are not structurally valid, and the diagnostic categories were not combined because a single row can be structurally invalid for more than one category. The deletion is therefore consistent with the special structural nature of the timestamp, duration, source-month and location-ID rules.

The approach intentionally maintained non-positive fare and distance rows where they were structurally sound. The final clean dataset had 2,151,347 records in the Invalid/Non-positive distance band and valid_distance and valid_fare specified whether that record was used for distance or fare-based averages. This resulted in demand counts remaining complete and prevented invalid numeric values from skewing the related analytical measures. The final dataset had no unmatched pickup lookup rows and was completely enriched in the Pickup-zone.

Outcome	Rows	Effect on analysis
Raw rows	89,892,322	100.000%
Structurally valid	89,291,810	99.332%
Structurally removed	600,512	0.668%
Final invalid/non-positive distance band	2,151,347	Retained for trip counts; excluded from positive-distance statistics
Unmatched pickup lookup	0	All structurally valid pickup IDs enriched

Table 5.2.1 Structural preprocessing outcomes

5.3 Hourly Demand Patterns

Hourly demand was calculated from all 89,291,810 structurally valid trips. The busiest hour for pickups was 18:00, during the entire period, with 6,384,616 trips. There was also a high concentration of journeys during the evening, with the next highest hours coming at 17:00 (6,062,737) and 19:00 (5,631,764). On the other side, the 04:00 only had 601,988 trips. The pattern continues with the later weekday/weekend analysis, although the overall hourly pattern on its own is not sufficient to determine the reason for the concentration in the week, such as commuting, tourism, nightlife, or other external factors.

Pickup hour	Trip count
18:00	6,384,616
17:00	6,062,737
19:00	5,631,764
16:00	5,458,603
15:00	5,327,825
04:00	601,988

Table 5.3.1 Selected pickup-hour trip counts for the full 2024–2025 period

5.4 Pickup Zones and Boroughs

Pickup activity was extremely localized. The greatest number of structurally valid pickups were found in Manhattan, totaling 77,669,263 points, about 86.98% of the full dataset. Queens was second with 8,489,722 trips (9.51%), followed by Brooklyn with 2,363,635 (2.65%) and the Bronx with 503,636 (0.56%). It can be seen from the borough counts that the vast majority of Yellow Taxi activity during the selected period came from Manhattan and Queens was still significant due to the airport zones.

Pickup borough	Trips	Share of clean rows
Manhattan	77,669,263	86.98%
Queens	8,489,722	9.51%
Brooklyn	2,363,635	2.65%
Bronx	503,636	0.56%

Pickup borough	Trips	Share of clean rows
Unknown	201,442	0.23%
N/A	45,621	0.05%
EWR	11,953	0.01%
Staten Island	6,538	0.01%

Table 5.4.1 Pickup trips by borough

At zone level, JFK Airport was the single most frequent pickup zone with 4,034,602 trips, narrowly ahead of Upper East Side South (4,001,341) and Midtown Center (3,973,590). The remainder of the top 20 was dominated by Manhattan zones, with LaGuardia Airport also appearing among the highest-ranked Queens locations. This concentration suggests that the airport and high activity density in Manhattan were two main drivers in the demand for Yellow Taxi pickups.

Rank	Borough	Pickup zone	Trips
1	Queens	JFK Airport	4,034,602
2	Manhattan	Upper East Side South	4,001,341
3	Manhattan	Midtown Center	3,973,590
4	Manhattan	Upper East Side North	3,568,124
5	Manhattan	Midtown East	2,902,776
6	Manhattan	Times Sq/Theatre District	2,889,401
7	Manhattan	Penn Station/Madison Sq West	2,883,630
8	Manhattan	Lincoln Square East	2,679,404
9	Queens	LaGuardia Airport	2,569,436
10	Manhattan	Murray Hill	2,454,289

Table 5.4.2 Top 10 pickup zones by trip count

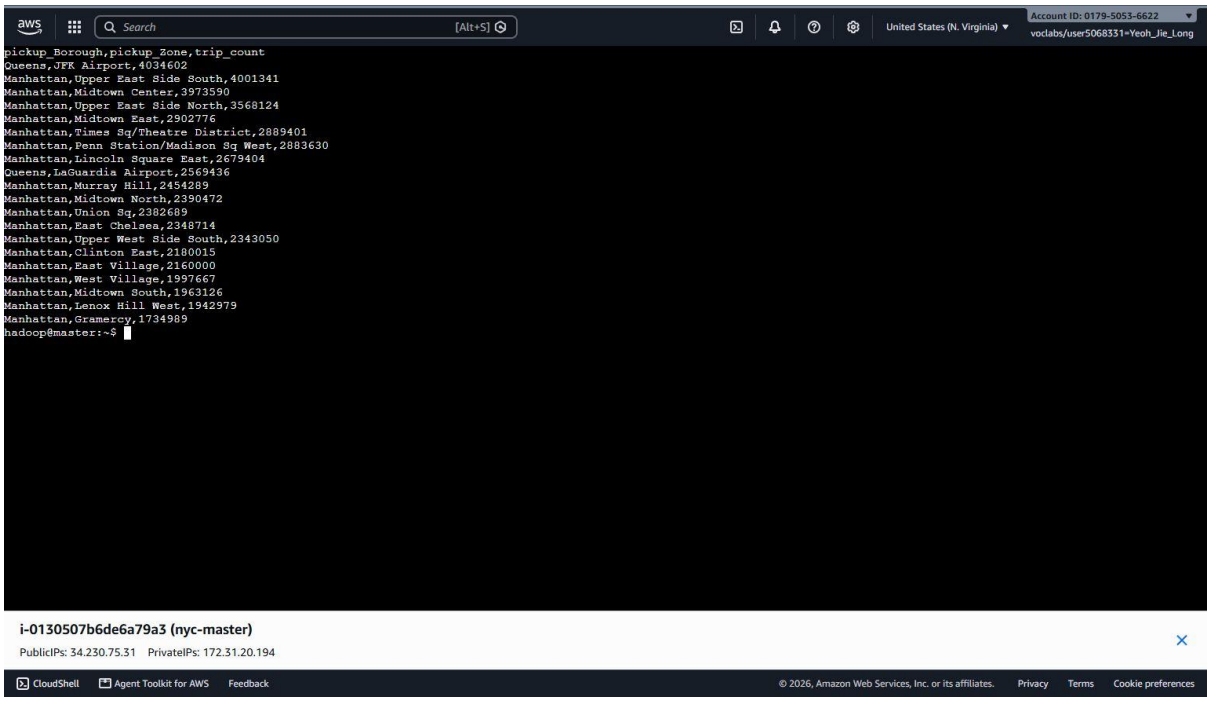


Figure 5.4.1 Spark top-20 pickup-zone result table for the full 2024-2025 dataset.

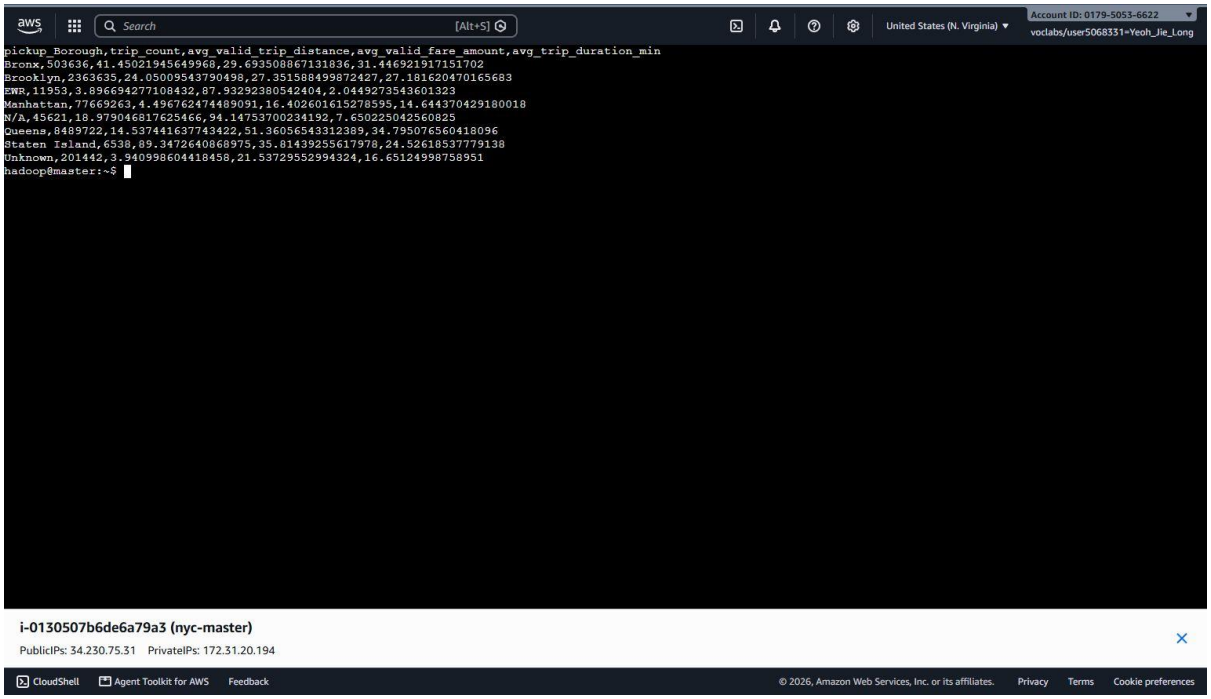


Figure 5.4.2 Spark pickup-borough summary for the full structurally valid dataset.

5.5 Fare Patterns by Distance Band

Distance band (miles)	Trip count	Avg fare (USD)	Median fare (USD)	Avg total (USD)	Avg duration (min)	Avg fare/mile (USD)
0-<2	46,357,514	10.76	10.00	17.79	9.6	26.03
2-<5	24,862,296	19.66	18.51	27.53	18.9	6.69
5-<10	8,862,332	34.15	33.10	46.18	27.8	4.82
10-<20	6,227,351	60.46	63.90	80.12	45.0	4.15
20+	830,970	92.45	70.00	116.13	57.1	3.72

Table 5.5.1 Distance-band fare, total amount and duration summary

Fare and duration increased strongly with distance. The average valid fare increases from USD 10.76 for trips below 2 miles to USD 92.45 for trips of 20 miles or more, while the average duration increased from 9.6 to 57.1 minutes. Besides that, the average total amount followed the same pattern, which increased from USD 17.79 to USD 116.13. These results show a clear positive association between trip length, travel time and the amount paid.

Besides that, the relationship was not linear when expressed per mile. Average fare per mile fell from USD 26.03 for trips below 2 miles to USD 3.72 for trips of 20 miles or more. In addition, fixed fare components and surcharges therefore have a much larger proportional effect on short trips. The longest-distance band also showed a notable gap between its mean fare of USD 92.45 and median fare of USD 70.00, which indicates that high-value trips and outliers raise the average.

A further 2,151,347 structurally valid records had invalid or non-positive distance values. They were retained for data-quality accounting but excluded from the positive-distance pattern table and charts. Fare-per-mile values should therefore be interpreted as descriptive measures rather than a direct pricing rule.

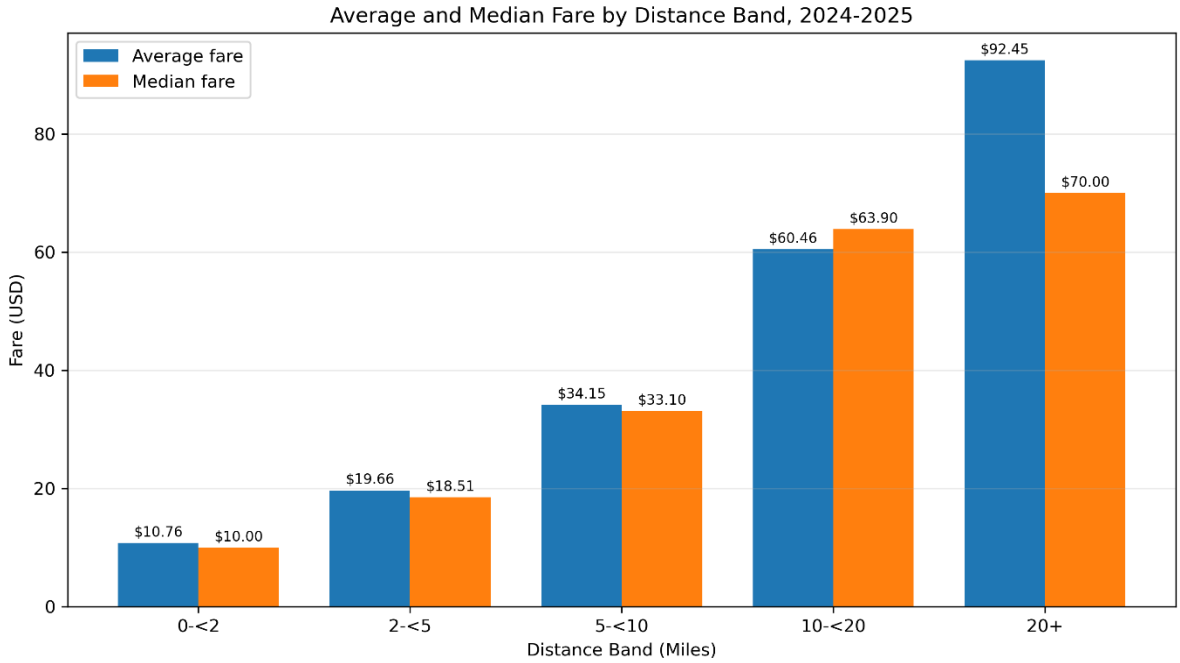


Figure 5.5.1 Average and median fare by distance band

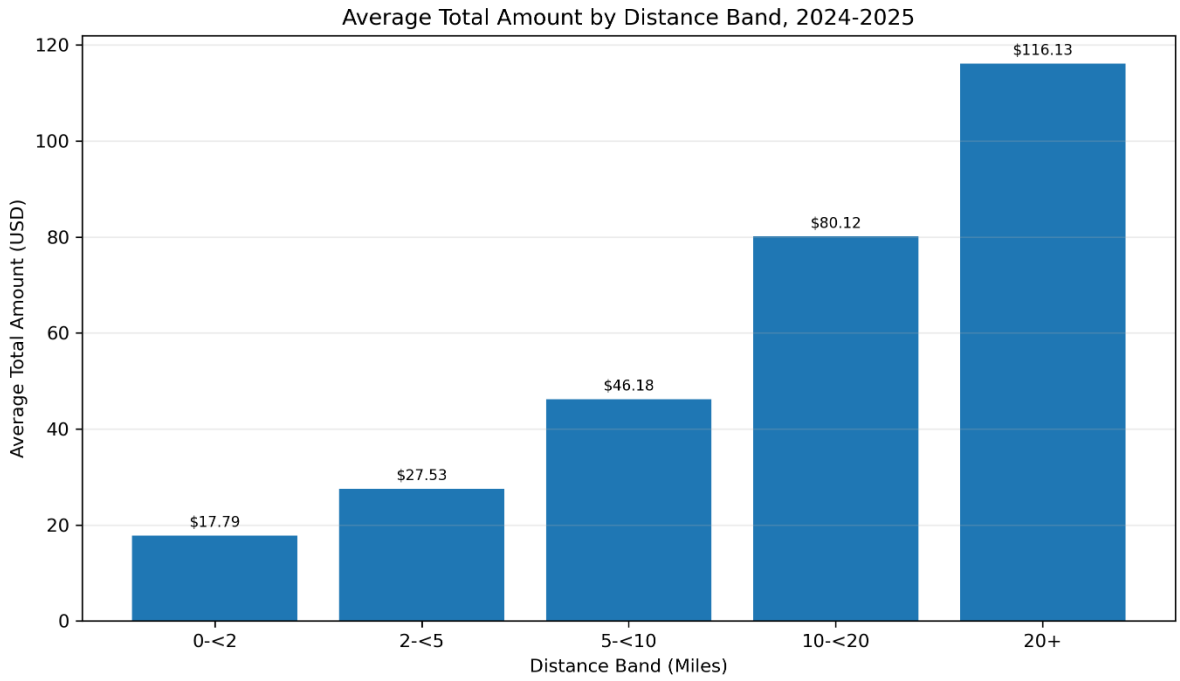


Figure 5.5.2 Average total amount by distance band

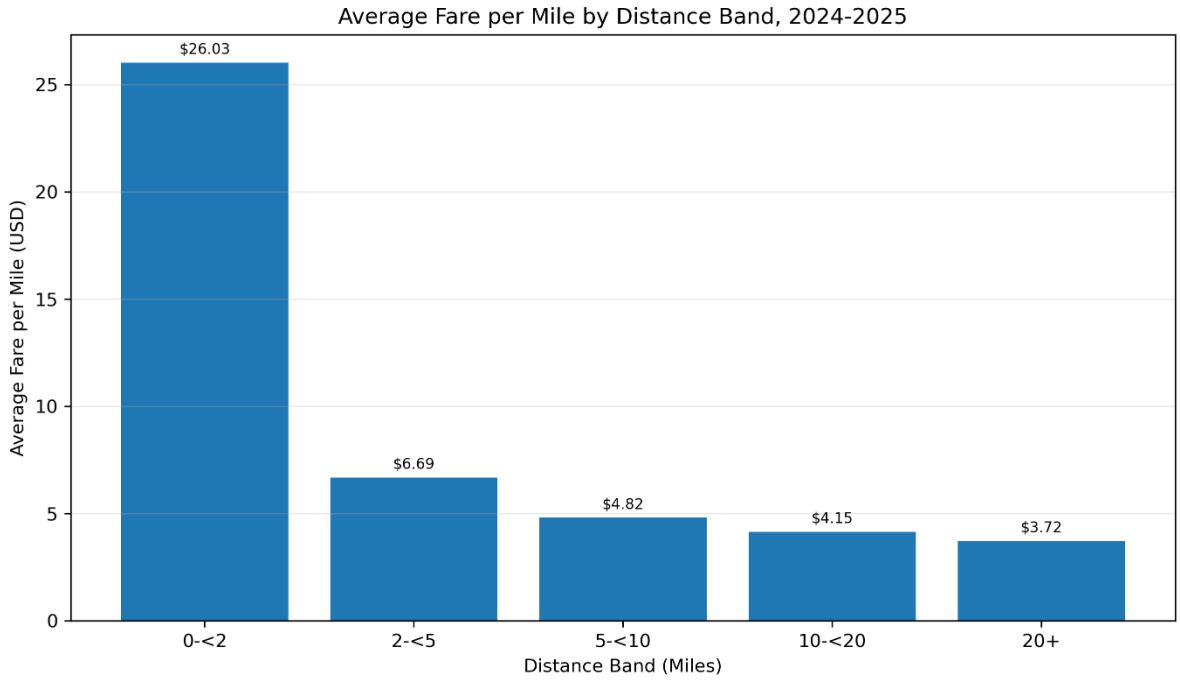


Figure 5.5.3 Average fare per mile by distance band

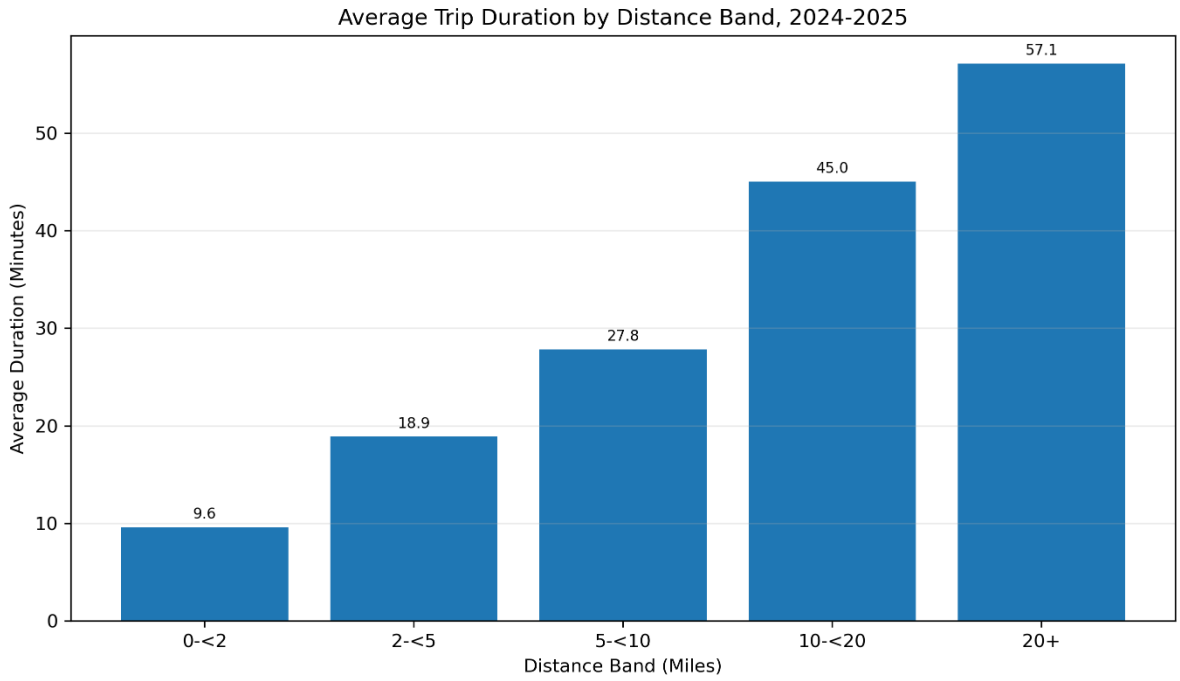


Figure 5.5.4 Average trip duration by distance band

5.6 Passenger-Count Patterns

Passenger count	Trip count	Share	Avg fare (USD)	Avg distance (miles)	Avg duration (min)
Missing	15,698,835	17.581%	21.09	19.10	18.05
0	656,204	0.735%	16.95	2.75	14.73
1	57,570,888	64.475%	19.22	3.46	16.43
2	10,457,033	11.711%	22.52	4.05	18.17
3	2,469,035	2.765%	22.44	3.92	18.01
4	1,634,036	1.830%	25.90	4.22	18.75
5	491,024	0.550%	18.61	3.22	15.46
6	314,330	0.352%	18.19	3.11	15.56
7+	425	< 0.000%	77.89	6.66	12.06

Table 5.6.1 Passenger-count summary

Single-passenger trips were the most recorded category, where contained 64.475% of all structurally valid records. Besides that, two-passenger trips were 11.711%, while counts from three to six passengers each represented much smaller shares. Furthermore, the missing

passenger counts remained substantial at 17.581%, and zero-passenger records accounted for 0.735%. Lastly, only 425 records were classified as seven or more passengers, so the very high average fare in that group should not be treated as representative.

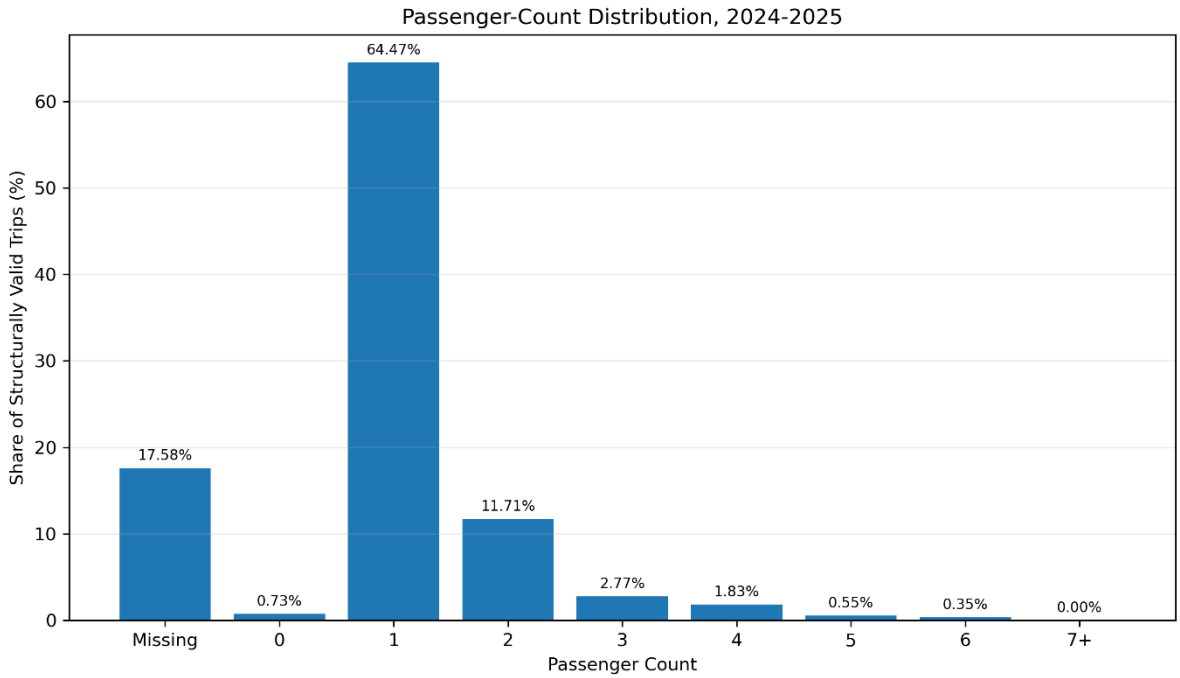


Figure 5.6.1 Passenger-count distribution

Passenger grouping	**Trip count**	**Share of valid 1-6**	**Avg fare (USD)**	**Avg distance (miles)**	**Avg duration (min)**
Single passenger	57,570,888	78.933%	19.22	3.46	16.43
Multiple passengers (2-6)	15,365,458	21.067%	22.64	4.00	18.07

Table 5.6.2 Single-versus-multiple passenger comparison

Among the records with passenger counts from one to six, single-passenger trips represented 78.933% compared to trips carrying two to six passengers. Multiple-passenger trips were associated with a higher average valid fare (USD 22.64 vs USD 19.22), a longer average valid distance (4.00 vs 3.46 miles) and a longer average duration (18.07 vs 16.43 minutes). These are descriptive associations only and do not demonstrate that passenger count causes higher fares or longer trips.

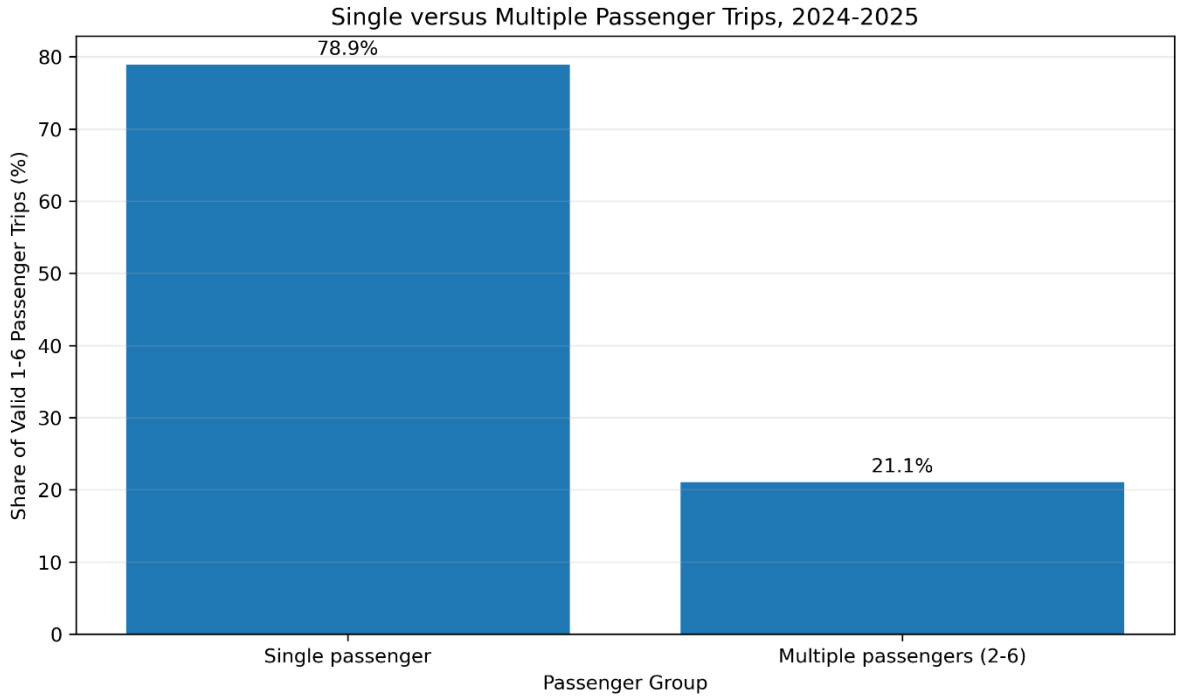


Figure 5.6.2 Single-versus-multiple passenger shares

5.7 Weekday versus Weekend

Day type	Trips	Avg daily trips	Peak hour	Peak-hour avg/day	Avg fare	Avg distance	Avg duration	Top pickup zone
Weekday	63,965,901	122,306	18:00	9,151	20.12	6.07	17.40	Midtown Center
Weekend	25,325,909	121,759	18:00	7,687	20.04	6.37	15.94	JFK Airport

Table 5.7.1 Weekday-versus-weekend summary

Average daily demand was very similar between weekdays and weekends, which approximate 122,306 trips per weekday and 121,759 trips per weekend day. They have the same peak hour at 18:00, but weekday demand was more concentrated around daytime and evening commuting periods. Meanwhile weekend activity was stronger after midnight.

Besides that, weekend trips were slightly longer on average compared to weekdays, while average duration was lower. Moreover, the average valid fare was similar at USD 20.04 on weekends and USD 20.12 on weekdays. Midtown Center was the leading weekday pickup zone, and JFK Airport was the leading weekend pickup zone. These findings describe different travel patterns but do not identify the causes of those differences.

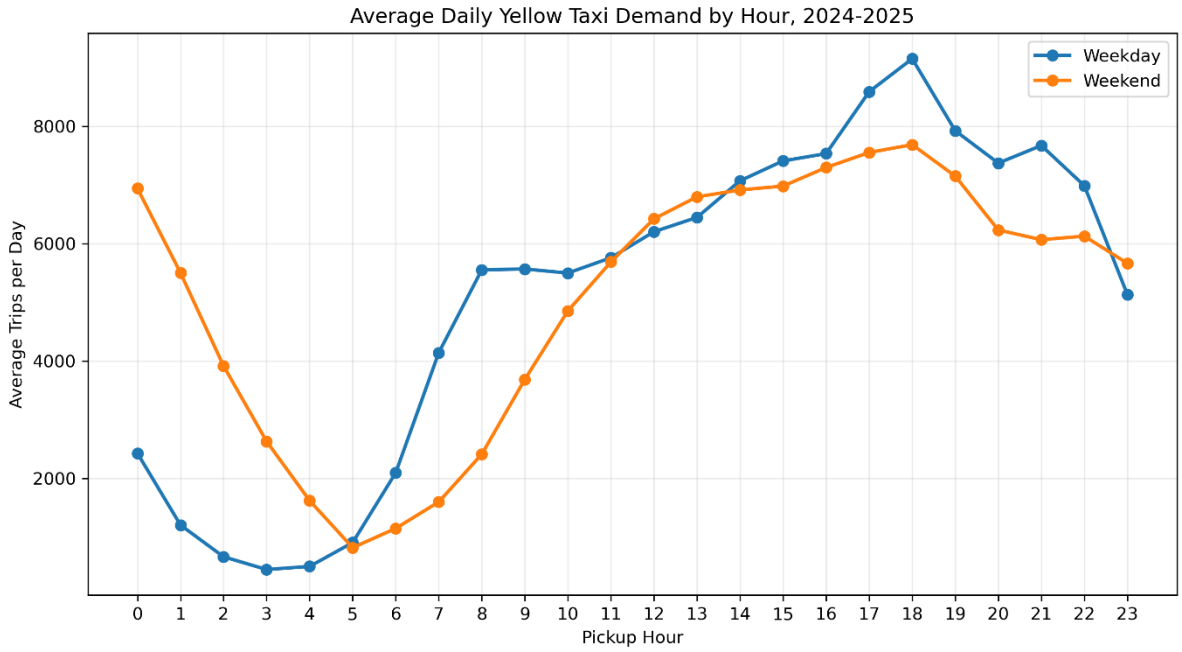


Figure 5.7.1 Average daily Yellow Taxi demand by hour: weekday versus weekend

5.8 2024 versus 2025 Comparison

Year	Valid trips	Avg daily trips	Peak month	Peak monthly trips	Avg valid fare	Avg total amount	CBD-fee charged trips
2024	41,132,445	112,384	October	3,830,854	19.96	28.75	0
2025	48,159,365	131,943	May	4,526,145	20.21	28.98	34,495,616

Table 5.8.1 Annual demand and fare comparison

The Yellow Taxi trips increased from 41,132,445 to 48,159,365 (17.1%) compared to 2024. Besides that, the average daily demand also increased from 112,384 to 131,943 trips. Other than that, October was the busiest month of 2024 and May was the busiest month of 20 25.

Moreover, the average valid fare increased from approximately USD 19.96 in 2024 to USD 20.21 in 2025. Meanwhile, the average total amount increased from USD 28.75 to USD 28.98. However, CBD congestion-fee was not available in the 2024. In 2025, 34,495,616 structurally valid trips recorded a positive CBD congestion fee, equivalent to approximately 71.6% of 2025 valid trips.

The year comparison is descriptive. Higher 2025 demand, fare and CBD-fee measures should not be interpreted as evidence that congestion pricing caused the observed changes because the dataset does not control for seasonality, tourism, weather, service availability, economic

conditions or other external factors.

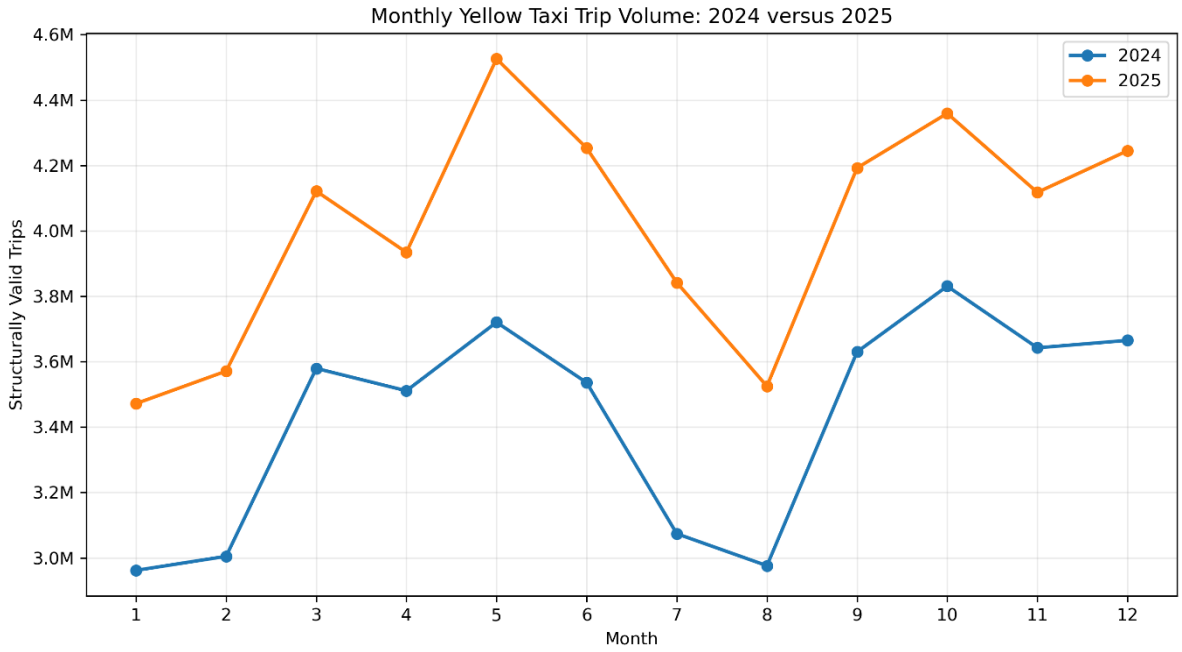


Figure 5.8.1 Monthly Yellow Taxi trip volume

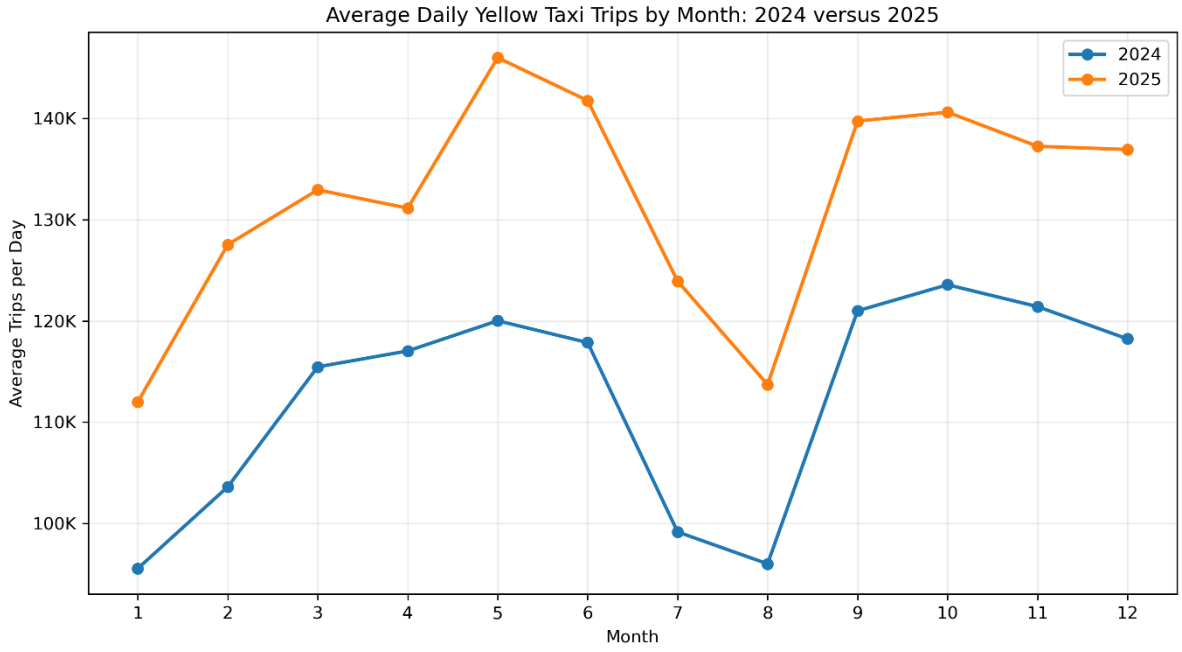


Figure 5.8.2 Average daily Yellow Taxi trips by month

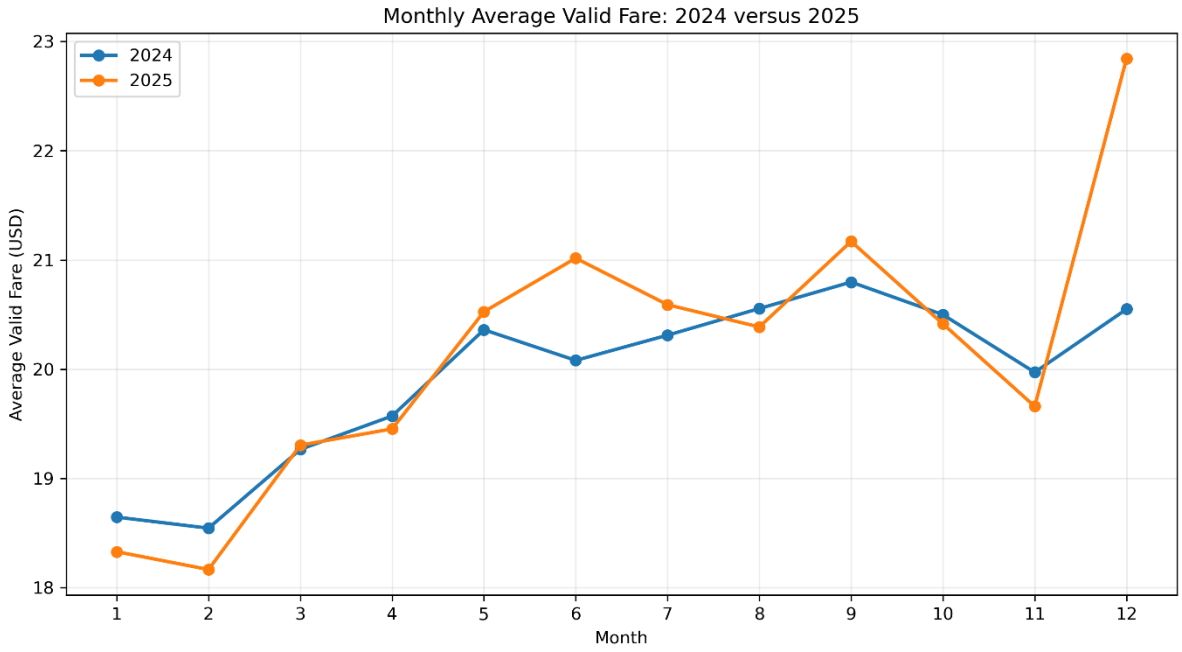


Figure 5.8.3 Monthly average valid fare

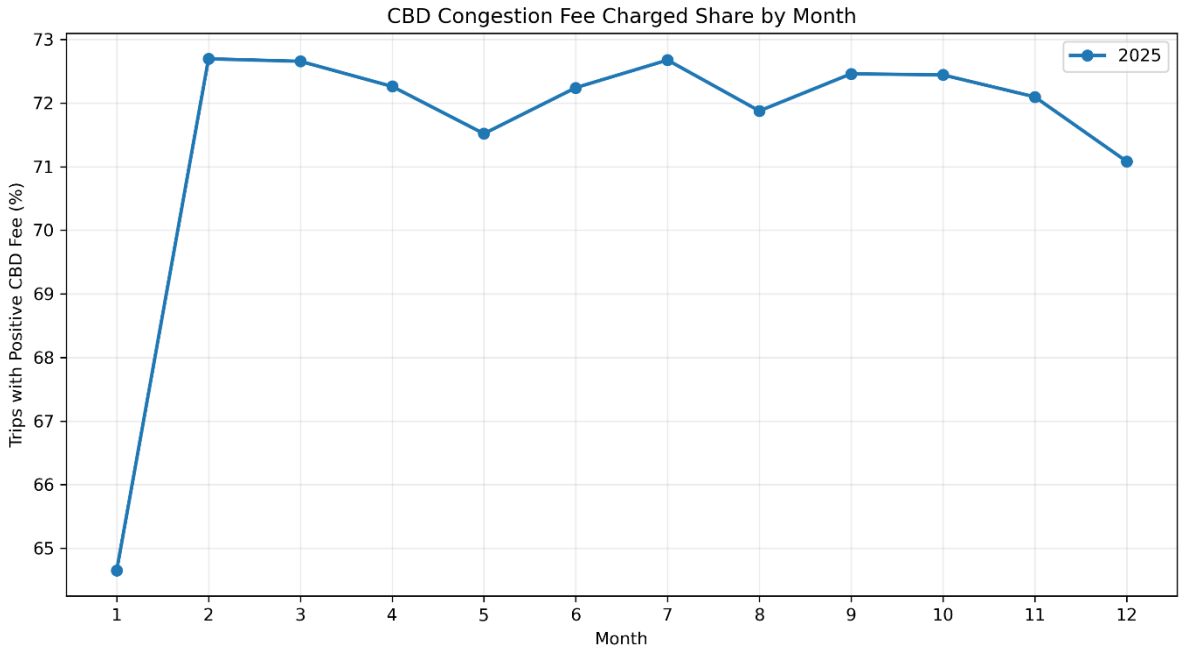


Figure 5.8.4 Share of trips with a positive CBD congestion fee

5.9 Integrated Findings

The results show that the Yellow Taxi activity between January 2024 and December 2025 was highly concentrated by time, location and trip characteristics. From 89,892,322 records, 89,291,810 trips remained structurally valid after preprocessing, while data-quality issues such as missing passenger counts, non-positive fare or distance values and timestamp anomalies required separate validity rules rather than removing every unusual record.

Temporal and geographic results showed clear concentration in Yellow Taxi demand. The busiest pickup hour was between 17:00 and 19:00, which showed a strong demand during evening. Besides that, Manhattan contributed approximately 86.98% of pickups, while JFK Airport was the leading individual pickup zone across the full period. Moreover, weekday and weekend had similar average daily totals, but their hourly demand and location patterns is different. Weekdays were more concentrated around daytime and evening periods and were led by Midtown Center. Weekend demand was stronger after midnight and JFK Airport was the leading weekend pickup zone.

Trip characteristics also showed consistent relationships. Higher distance bands were associated with higher average fare, total amount and trip duration, while average fare per mile decreased as distance increased. Single-passenger trips represented the largest recorded passenger category, although the substantial amount of missing passenger-count data limits the strength of conclusions based on this variable. The year comparison further showed that 2025 recorded 48,159,365 structurally valid trips compared with 41,132,445 in 2024, approximately 17.1% more, together with a slightly higher average valid fare. The CBD congestion-fee field was available in 2025 and was positive for approximately 71.6% of valid 2025 trips. These differences are descriptive and should not be interpreted as evidence that congestion pricing caused the observed changes.

Overall, the analytical questions were answered using consistent preprocessing and aggregation rules across the 24-month dataset. The findings demonstrate substantial temporal and geographic concentration in Yellow Taxi activity, systematic associations between distance and fare characteristics, differences in passenger and weekday/weekend travel patterns, and higher recorded trip activity in 2025. The following table summarises how each research question was addressed by the project results.

6.0 Comparison

6.1 Correctness Comparison

Correctness validation was limited to January 2024 to allow Spark and pandas to be compared based on the same source month, preprocessing rules and Taxi Zone Lookup. Both implemented programs selected the same number of structurally valid records in the set of 2,964,624 raw January records, namely 2,961,858 records. The matched picking-up zone outcome also revealed that JFK Airport is the top picking-up zone in both implementations, with 145,025 trips. These exact matches verify that the logic and picking up locations for structural cleaning and structural validation remain consistent for the common validation month.

The pandas baseline for January also showed 18:00 as the peak hour for pickups, at 212,616 trips, and average valid fare values by distance band. Equivalent January-specific

Spark outputs were not saved in the correctness file, so are not listed as cross-platform equality checks, but as pandas results. The overall scalability comparison is performed independently with the matched 1, 3, 6, 12 and 24-month top-pickup-zone workloads in Section 6.2.

Validation item	Spark result	pandas result	Status
Structurally valid rows	2,961,858	2,961,858	PASS – exact
Top pickup zone	JFK Airport (145,025 trips)	JFK Airport (145,025 trips)	PASS – exact

Table 6.1.1 January 2024 Spark-versus-pandas correctness validation

6.2 Runtime and Scalability

The median times for pandas and Spark for the matched top-pickup-zone workload were 0.464, 1.368, 3.040, 5.937 and 12.814 seconds, and 15.635, 28.578, 48.815, 93.906 and 191.776 seconds respectively. On the small two-node cluster, pandas was faster for this particular workload because of the small cluster, but Spark had extra overhead for YARN and distributed-execution. The comparisons of runtimes should thus be taken as performance difference between execution environments, but not as a hardware-to-hardware speedup.

Months	**Valid rows**	**Spark median runtime (s)**	**pandas runtime (s)**	**Validation**
1	2,961,858	**15.635**	**0.464**	PASS
3	9,546,211	**28.578**	**1.368**	PASS
6	20,313,815	**48.815**	**3.040**	PASS
12	41,132,445	**93.906**	**5.937**	PASS
24	89,291,810	**191.776**	**12.814**	PASS

Table 6.2.1 Matched Spark-versus-pandas top-pickup-zone scalability benchmark

In the tested environments, the median time for aggregation of the pandas was faster than the Spark median time for every scale tested. This does not mean panda is generally quicker as the platforms were set up on different hardware and architectures. Instead, the Spark result shows that using distributed HDFS/YARN execution on a very small cloud cluster, the same validated workload can be executed successfully with recovery of an executor failure during an excluded 24-month period.

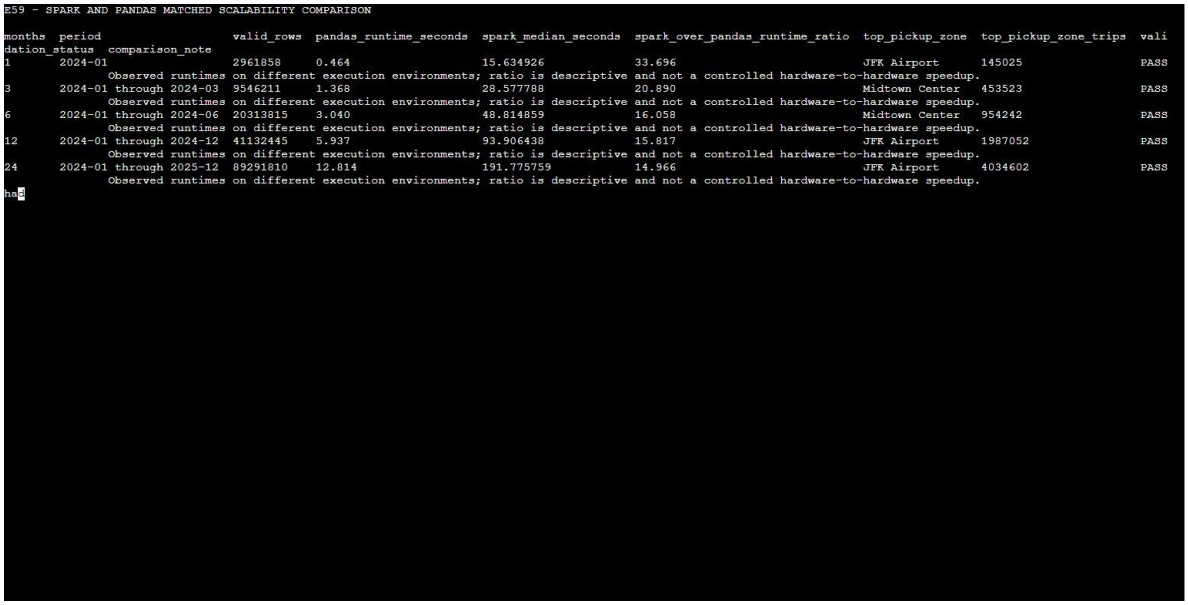


Figure 6.2.1 Matched Spark-versus-pandas scalability comparison with exact validation across the 1, 3, 6, 12 and 24-month scales.

6.3 Memory and Resource Feasibility

The feasibility of the resources was assessed based on the observed pandas process memory during the matched benchmark and the actual Spark cluster configuration. The maximum resident set size (RSS) for pandas was measured at each of the scales. For Spark, a directly comparable peak-RAM measurement was not collected; therefore, the actual cluster and executor configuration is reported instead of estimating a memory value.

Scale (months)	Valid rows	Peak RSS (MB)	Status
1	2,961,858	465.67	SUCCESS
3	9,546,211	485.34	SUCCESS
6	20,313,815	487.79	SUCCESS
12	41,132,445	489.34	SUCCESS
24	89,291,810	513.42	SUCCESS

Table 6.3.1 pandas memory observation by benchmark scale

Table 6.3.1 indicates that all five benchmark scales were completed by pandas on the local computer. The observed peak RSS grew from 465.67 MB for the first month to 513.42 MB for the full 24 months of 89,291,810 structurally valid rows. The increase was relatively small since the benchmark was processing the monthly files one at a time and it was only leaving behind the compact aggregation results, not the full dataset in one big DataFrame.

Resource aspect	pandas	Spark
Execution environment	Local single-machine Python	Two-node EC2 cluster with HDFS/YARN
Compute structure	Single local process	1 master + 1 worker
Executor configuration	N/A	1 executor, 1 executor core
Executor memory	N/A	512 MB
Executor overhead	N/A	256 MB
Driver memory	N/A	768 MB
Measured peak memory	465.67-513.42 MB RSS	N/A - not directly measured
24-month completion	SUCCESS	SUCCESS
Full valid rows processed	89,291,810	89,291,810

Table 6.3.2 pandas and Spark resource-feasibility comparison

The Spark benchmark used a two-node t3.small EC2 environment consisting of one master and one worker under HDFS/YARN. The matched benchmark was set to have one executor, one executor core, 512 MB executor memory, 256 MB executor overhead, and 768 MB driver memory. No directly comparable Spark peak-RAM value was collected, and is therefore reported as N/A instead of estimated.

With monthly files processed in a sequential manner, both platforms were able to handle the full 24 month workload with 89,291,810 structurally valid rows. Spark showed that it is feasible to implement full data execution with distributed HDFS/YARN even with a small cluster configuration. Thus, it's not possible to directly compare the resource usage efficiency of the two platforms since they have different execution architectures, and just pandas is measured with peak-RSS.

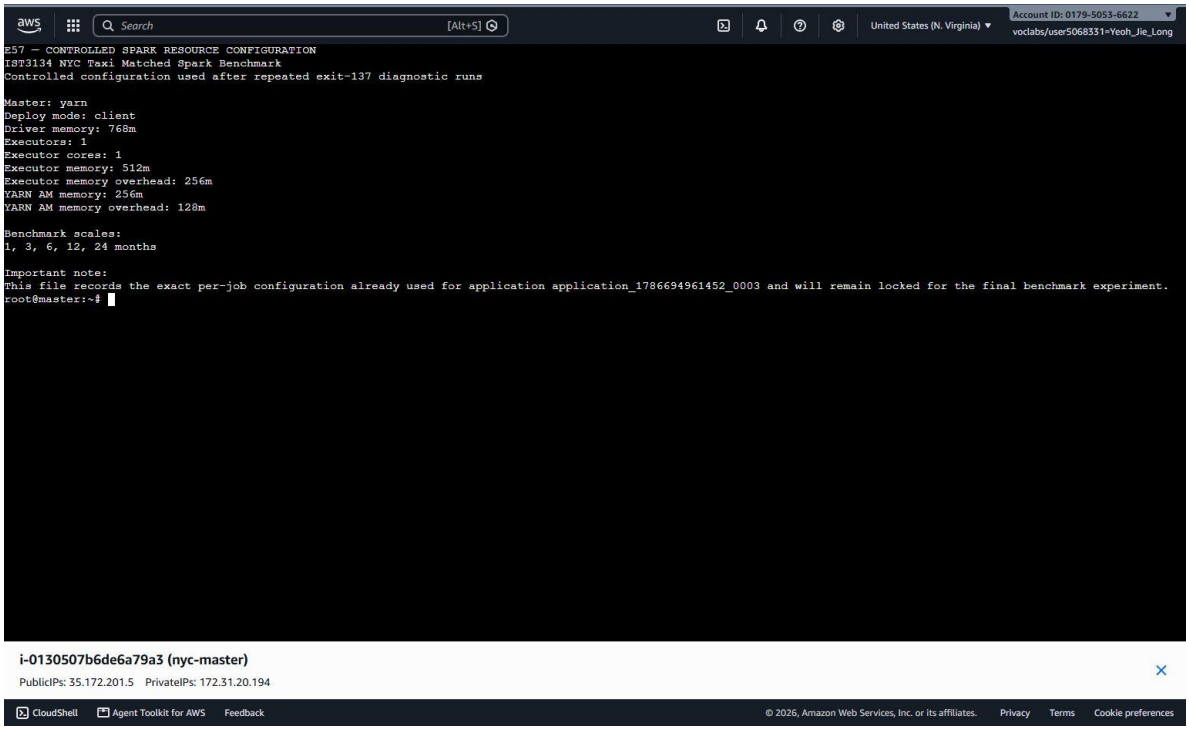


Figure 6.3.1 Controlled Spark resource configuration used for the matched scalability benchmark.

6.4 Development and Operational Complexity

Aspect	pandas	Spark/AWS-HDFS-YARN
Environment setup	Local Python environment and packages	AWS Academy, S3, EC2, HDFS, YARN and Spark configuration
Data storage	Local monthly Parquet files	Private S3 source layer plus HDFS raw, processed and result paths
Execution model	Single-machine, sequential monthly aggregation	Distributed lazy evaluation across executors
Debugging	Fast local reruns and direct Python errors	Requires cluster/log/partition/shuffle diagnosis
Scalability controls	Chunk/month-wise processing and memory management	Partitions, executors, shuffle, adaptive execution
Fault tolerance	Limited to local process/file recovery	Distributed recomputation and cluster mechanisms
Operational cost	No cloud runtime charge	Cloud credit/cost and cluster lifecycle management

Table 6.4.1 Development and operational complexity comparison

pandas needed less infrastructure and was easier to rerun and debug locally. Spark required additional configuration for S3, EC2, HDFS, YARN, cluster resources and distributed execution. However, that additional operational complexity provided shared distributed storage, partition-aware execution and recovery mechanisms that were observed during the project. This is because the code length and one wall-clock result are not necessarily the most relevant factors for choosing which one to use, depending on the size of the workload, concurrency, resilience and other operation requirements.

6.5 Suitable Use Cases

Scenario	pandas suitability	Spark suitability
Small / exploratory subsets	Preferred when data fits comfortably on one machine and rapid iteration is important	Usually unnecessary because cluster startup and scheduling overhead are not justified
Medium sequential aggregations	Suitable when month-wise/chunk-wise processing keeps memory controlled	Useful when the same data is already in the cloud or multiple distributed operations are required
Full 24-month workload in this project	Feasible for the implemented aggregate analyses; 24-month benchmark completed successfully	Suitable for the complete cloud pipeline, centralised processed data and repeated distributed analytics
Larger / more complex future workloads	May become constrained by local CPU, RAM and storage	Preferred when scale, concurrency, fault tolerance and distributed transformations become important

Table 6.5.1 Suitable use cases

For this project, pandas proved fully capable of the selected 24-month aggregate workload when implemented sequentially, and it provided a simple platform for validation and visualisation. Spark remained valuable as the main cloud Big Data platform because it processed the complete centralised dataset using distributed storage and execution. The results therefore do not support the claim that one platform is universally faster; instead, pandas is attractive for efficient local analysis, while Spark is more suitable when distributed scalability, shared cloud storage and more demanding workloads are required.

7.0 Discussion

7.1 Interpretation of Findings

The results are combined and exhibit noticeable temporal, geographic and fare-related patterns in the 2024-2025 Yellow Taxi data. The demand was peaked in the evening, with 18:00 being the most crowded hour of picking up during the whole of the data set. Geographical pickup activity was also focused, with JFK Airport being the top spot in the full-period zone rank, and the top spots in the weekday and weekend results were different, with different hourly pickup profiles.

Fare and duration showed strong association with distance, but the distance decreased as fare per mile increased. The big fare-per-mile figure for short trips suggests that the fixed elements of taxi pricing play a bigger part at brief distances. The number of passengers was mostly represented by single-passenger tests, but a significant missing number of passengers also reduced the value of conclusions about this variable.

The 2025 data contained substantially more structurally valid trips than 2024 and showed a slightly higher annual average valid fare. A CBD congestion-fee field was also introduced in 2025 and was positive for a high proportion of trips. The observations are descriptive. The results should not be viewed as evidence of the effect of congestion pricing on the changes as the data does not factor in seasonality, economic factors, tourism, weather, service availability, or other external factors.

The platform comparison yielded a valuable practical result. A carefully designed pandas workflow could process the entire 24-month aggregation workload locally through sequential processing of files, while Spark independently generates matching analytical results through distributed cloud execution. Agreement on the primary correctness measures validates that the analytical definitions were not altered by the platform selection.The main distinction lies in execution architecture, operational complexity and scalability for future or more demanding workloads.

7.2 Advantages of Big Data Analytics in the Cloud

The AWS/Spark pipeline was beneficial beyond the shortest benchmark time. Amazon S3 kept a private durable source copy, HDFS provided a distributed execution layer, YARN managed to schedule the Spark application, and partitioned Parquet enabled the entire dataset of 89.29 million rows in a cleaned dataset to be processed without the need to push it into a single local in-memory table. In addition, different analytical expressions were used for Spark DataFrames and Spark SQL, and yielded consistent results. On the other hand, during execution, YARN/Spark demonstrated recovery from both an executor heartbeat problem in preprocessing as well as an executor-exit event observed during the 24-month matched

benchmark, highlighting the practical use of distributed recovery mechanisms.

The matched benchmark demonstrates that cloud Big Data platforms cannot only be justified based on “faster wall-clock execution”. On the tested two-node t3.small cluster, YARN scheduling and distributed coordination added substantial overhead for the compact top-zone aggregation, while pandas completed the same workload faster locally. The primary cloud benefit from this project was to have full data distributed processing, shared storage, partitioned execution, reproducibility/resilience, and not guaranteed speed for all queries.

7.3 Limitations and Threats to Validity

There are a few limitations to the technical comparison. During the matched benchmark, spark was deployed on a t3.small two node Academy cluster with one worker, and pandas was deployed on different local hardware. There was no measurement performed that can be directly compared to Spark peak-RAM, so there is no claim to RAM-efficiency. Spark timings are also affected by JVM start up, YARN scheduling, caching and storage effects for short Sparks. Due to the failure of the EMR attempt and Academy/Vocareum constraints, the test of a larger managed cluster was limited. Following the latter EC2 restart, HDFS reported that its blocks were under-replicated, with one live DataNode in the final environment but no blocks being missing or corrupted.

The data also does not give enough information to interpret. Some trips have quality issues, a significant proportion of trips have missing passenger_count data, cash tips are not present in the data, and the schemas in 2024 and 2025 differ due to the addition of cbd_congestion_fee in 2025. The data presented is observational, and year-to-year variations cannot confirm that congestion pricing led to a change in demand or fare behaviour.

7.4 Recommendations and Future Improvements

Future work should be to repeat the matched benchmark under controlled settings, repeated cold/cached runs, repeated warm runs and, if possible, under similar hardware. A larger Spark cluster with more worker nodes would be a more interesting scale out test, and if it's safe in Academy to have the cluster in a stable state, the manual administration of Hadoop/YARN would be minimized in a managed EMR environment. Spark UI and YARN metrics should also be collected for executor memory, shuffle read/write, spill and task-level behaviour. The tuning that follows can consider the memory of the executors, shuffle the partitions or shuffle the size of the partitions before applying further years of data or more complex analytical workloads.

8.0 Conclusion

8.1 Achievement of Objectives

January 2024 correctness validation confirmed exact agreement for the structurally valid row count and leading pickup zone, while the matched 1, 3, 6, 12 and 24-month scalability benchmark achieved exact validation for valid rows, top pickup zone and top-zone trip count at every scale.

Objective	Status	Evidence
1. Build a reproducible cloud Spark pipeline	Achieved	24 raw monthly files stored privately in S3, staged to HDFS and processed on the successful EC2 Hadoop/YARN/Spark cluster.
2. Profile, validate and clean the source data	Achieved	24 months profiled; 89,892,322 raw rows reduced to 89,291,810 structurally valid rows using documented rules.
3. Convert location IDs to meaningful pickup areas	Achieved	Taxi Zone Lookup contained 265 valid IDs; pickup left join produced 0 unmatched rows.
4. Analyse temporal, location, distance, passenger and period patterns	Achieved	Spark A1-A7, Spark SQL validation and the full-period analytical summaries were completed.
5. Compare Spark and pandas	Achieved	January 2024 correctness checks passed, and the matched 1, 3, 6, 12 and 24-month scalability benchmark passed exact validation at every scale.
6. Publish code, instructions, small outputs and evidence online	Achieved	The jie-long-spark branch contains the core Spark scripts, small result artifacts, physical plans, execution log and evidence index; raw data and credentials are excluded.

Table 8.1.1 Achievement of project objectives

8.2 Final Conclusion

It read the 89,892,322 raw NYC Yellow Taxi data points from January 2024 to December 2025, and in both frameworks the Spark and the pandas, applied a common preprocessing framework and filtered out structurally invalid trips, which left 89,291,810 trips. Combined analysis revealed that the busiest hour for pickups was 18:00 and the top pickup zone was JFK Airport throughout the period. Fare and trip length were typically found to rise with distance traveled, per mile fare typically fell with distance traveled. Single-passenger trips made up the largest passenger group, while different hourly demand patterns and primary pick-up locations emerged from the weekday vs. weekend comparison. Together, the Spark-generated aggregates and the pandas-based detailed summaries and visualisations provided a consistent view of temporal demand, geographic concentration, fare behaviour and passenger patterns across the two-year dataset.

The year comparison revealed 48,159,365 structurally valid trips in 2025, with an increase over 2024 of about 17.1%. The average valid fare was also slightly larger for 2025, and the CBD congestion-fee field, which was not available in the 2024 files recorded a positive value for about 71.6% of valid 2025 trips. These differences were not explained causally but only descriptively. To ensure cross-platform correctness, Spark and pandas were validated against the same dataset – the one from January 2024 – and then compared to see if they agree on the structurally valid trip count, 2,961,858, and the top pickup destination, the JFK Airport with 145,025 trips. The individual matched scalability benchmark then validated that valid-row counts, top pickup zones and top-zone trip counts matched between the 1, 3, 6, 12 and 24-month workloads. This separation is achieved in the methodology of the correctness validation and the scalability evaluation and is reflected in the analytical consistency of both platforms.

Results indicate that panda achieved excellent performance for this highly structured local aggregation workload: the runtime rose from 0.464 seconds for one month to 12.814 seconds for 24 months, and the peak RSS usage was 465.67 MB and 513.42 MB respectively. Spark completed the same matched workload on the two-node HDFS/YARN cluster with median runtimes of 15.635, 28.578, 48.815, 93.906 and 191.776 seconds for the five scales. Pandas was quicker for this particular benchmark on the machines tested, but it wasn't a hardware-to-hardware speed test, so this shouldn't be used as a basis for comparison. Spark continued to be valuable as the basic cloud Big Data platform, offering the capability to handle the entire distributed processing and analytical pipeline, as well as storage in S3 and HDFS, processing via Spark SQL, partitioned execution and fault recovery. Taking everything together, the results indicate that pandas and Spark are complementary, not better suited alternatives: pandas is suitable for efficient local analysis and visualisation for which the memory consumption can be managed, while Spark is more appropriate when distributed storage, scalability, resilience and the wider cloud-processing requirements are important.

9.0 Individual Reflection

9.1 Yeoh Jie Long

My primary contribution to this assignment was the AWS and Spark side of things: dataset manifest verification, dataset S3 source layer organisation, Hadoop/HDFS/YARN environment setup, PySpark profiling/preprocessing, running DataFrame and Spark SQL analyses, benchmarking the workload and collecting execution evidence and keeping the Spark GitHub branch maintained. This assignment was to bring together the concepts of Linux, Hadoop, HDFS, YARN, and Spark from Labs projects to form an end-to-end pipeline rather than a series of isolated exercises.

The biggest challenge was that the planned managed EMR cluster did not become

usable in AWS Academy. I was trying to create the EMR 7.13.0 cluster, but it ran into Starting and finally shut down with INTERNAL_ERROR. I go with the course manual route and created a self managed EC2 Hadoop/YARN cluster. The initial t3.micro configuration was found to be too small to support stable Spark runs and the master/Worker configuration was resized to t3.small and the Hadoop/YARN services were re-validated. This is a lesson I learned that when you were planning your managed service you had to consider the real environment of the clouds rather than assuming that this service was always available.

Another practical problem was that the course AMI did not include the AWS CLI, S3A libraries or boto3. I used a Python standard-library Signature Version 4 transfer method to stage and verify all 24 monthly files and the Taxi Zone Lookup in HDFS, after which the temporary credentials were removed. I also learned during the pre-processing why pre-processing the structural validity should not be part of the analysis-specific validity; non-positive fare or distance did not necessarily mean that a trip was unusable for demand analysis, so valid_fare and valid_distance flags were able to preserve useful information while keeping statistics appropriate.

The distributed execution behaviour became more meaningful when failures occurred. A December preprocessing executor heartbeat problem was recovered by YARN without changing the data manually, and the final clean Parquet count still matched 89,291,810 rows. Later, a 24-month matched benchmark observation had an exit-executor of 137. I didn't hide or treat that timing as a performance result, I logged a clean one, and I only used a clean three timings to calculate the final median. These events made Spark fault tolerance a reality rather than just a theory.

The final comparison also changed how I understand performance evaluation. The earlier Spark benchmark and the pandas benchmark were not the same workload, and so cannot be used as a current scalability comparison. Finally, for a scalability comparison, the workload was scaled up and down on both implementations and the results were validated at 1, 3, 6, 12, and 24 months, and pandas was the faster of the two in the test environments, contrary to the simple assumption that Spark will always be faster. I learned that a valid benchmark must prioritise identical logic, repeated observations and honest interpretation over trying to prove a preferred platform. Spark was still useful, since it enabled the entire distributed pipeline and common storage and recovery infrastructure.

We also had to ensure that the preprocessing and comparison definitions are consistent across Spark and pandas. My role was to provide traceable Spark outputs, benchmark configuration and evidence that could be checked independently on the pandas side. Overall, the project improved my confidence in cloud troubleshooting, distributed-data design, reproducibility and critical performance interpretation, especially when the real

execution environment did not behave exactly like the original plan.

9.2 Ooi Jin Yon

Throughout this project, my main focus were on the pandas implementation, correctness validation, scalability benchmarking, visualisation and analytical results. One of the most important things I learned was the way to process a large dataset efficiently on a single machine. Instead of loading all Parquet files into one DataFrame, I let the files process one by one and retained only the required results. This technique helped to reduce unnecessary memory usage and allowed pandas to complete it successfully.

Besides that, one challenge I met was to ensure that the pandas and Spark used the same preprocessing rules and analytical definitions. As both tools process data in a different way, I had to check the filtering conditions, valid-record criteria and calculation methods used in both method to ensure they was the same. This was important to ensure that any differences in the results were caused by the processing approach. Through this process, I had learned the importance of maintaining consistent data-processing rules when comparing different big data technologies.

In addition, the scalability benchmark also improved my understanding of the differences between pandas and Spark. pandas had successfully completed the 1, 3, 6, 12 and 24-month workloads while maintaining manageable memory usage. Although pandas recorded lower runtime for the selected aggregation, but this does not automatically mean it is better than Spark. Spark provides distributed execution, fault recovery and better support for larger or more complex workloads compared to pandas.

Overall, this project help to improve my skills in pandas, data preprocessing, performance benchmarking and visualisation. Besides that, it also improved my understanding of how to choose an appropriate processing platform based on workload size, available resources and analytical requirements.

10.0 Reference

Apache Software Foundation. (2024). *Apache Parquet*. https://parquet.apache.org/

Armbrust, M., Xin, R. S., Lian, C., Huai, Y., Liu, D., Bradley, J. K., Meng, X., Kaftan, T., Franklin, M. J., Ghodsi, A., & Zaharia, M. (2015). Spark SQL: Relational data processing in Spark. *Proceedings of the 2015 ACM SIGMOD International Conference on Management of Data*, 1383–1394. https://doi.org/10.1145/2723372.2742797

Salloum, S., Dautov, R., Chen, X., Peng, P. X., & Huang, J. Z. (2016). Big data analytics on Apache Spark. *International Journal of Data Science and Analytics*, *1*(3), 145-164. https://doi.org/10.1007/s41060-016-0027-9

Zeng, X., Hui, Y., Shen, J., Pavlo, A., McKinney, W., & Zhang, H. (2023). *An empirical evaluation of columnar storage formats*. arXiv. https://doi.org/10.48550/arXiv.2304.05028

11.0 Appendix

https://github.com/ItzSeir/Small-Data-Analytic